

DIGITAL IMAGE PROCESSING

A practical introduction with Python and OpenCV

2026-04-13 - Jérôme Landré – jerome.landre@ju.se

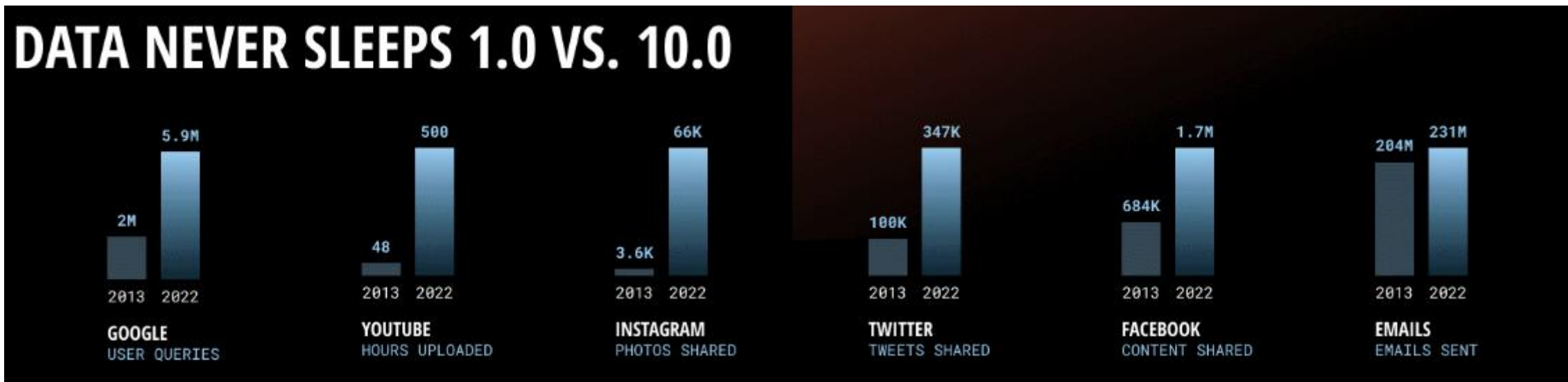
OUTLINE

1. What is Digital Image Processing?
2. What is an image?
3. Methodology for Image Processing
 1. Images sources
 2. Fixed or moving camera
 3. Type of images
 4. Color images or not...
 5. Background removal
 6. Edges and regions
 7. Region of interest
 8. Extract information
 9. Machine Learning
 10. Tools
 11. Other (image) sensors

1. What is Digital Image Processing?

DIGITAL IMAGE PROCESSING

- More and more digital images are produced every day from our phones, cameras, computers...
- Every MINUTE on the Internet:



DIGITAL IMAGE PROCESSING

- “Digital image processing is the use of a digital computer to process digital images through an algorithm.” from https://en.wikipedia.org/wiki/Digital_image_processing



DIGITAL IMAGE PROCESSING

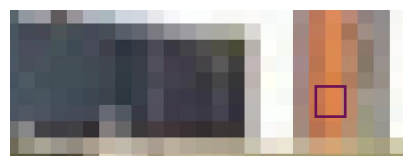
- The goal is to extract useful information from images in order to detect/track/find/identify/sort/classify objects in one or more image(s).
- Applications of Digital Image Processing:
 - Image restoration,
 - Robot vision,
 - Medical imaging,
 - Satellite imaging,
 - Pattern recognition,
 - Image/Video analysis...

2. What is a digital image?

WHAT IS A DIGITAL IMAGE?

WHAT IS A DIGITAL IMAGE?

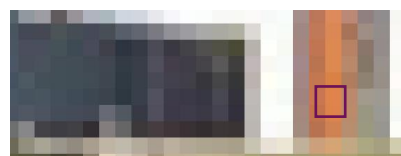
- A 2D array of pixels



#RGB: 755844

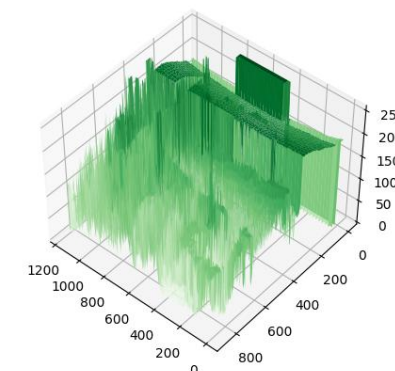
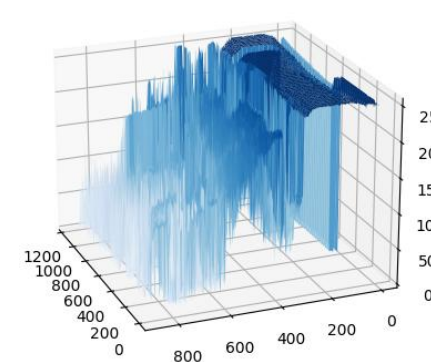
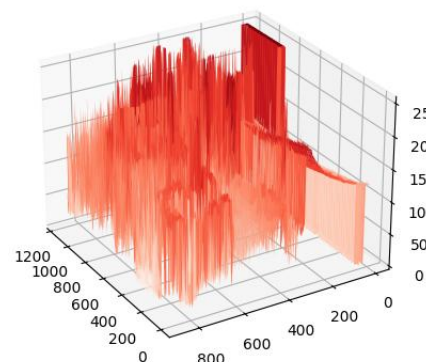
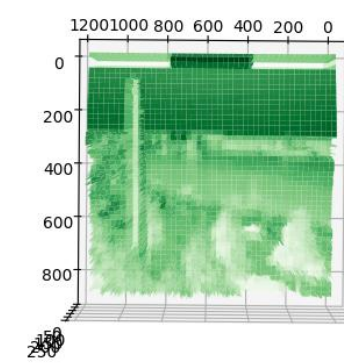
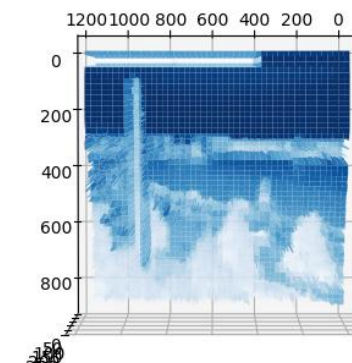
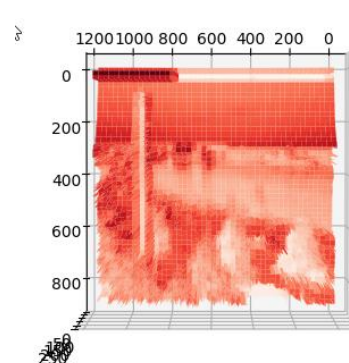
WHAT IS A DIGITAL IMAGE?

- A 2D array of pixels



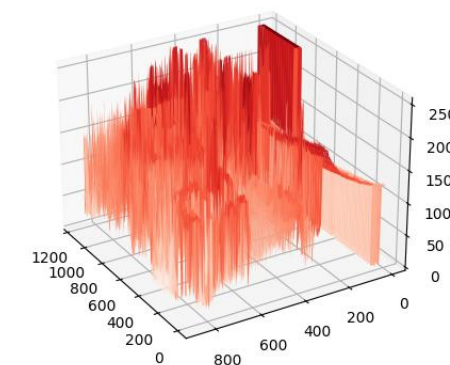
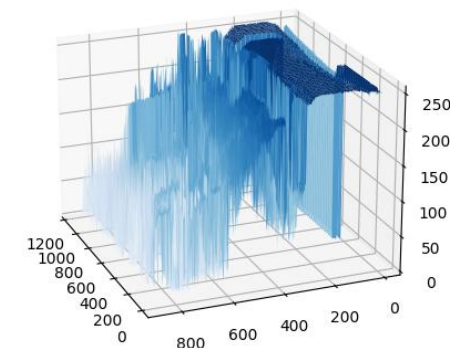
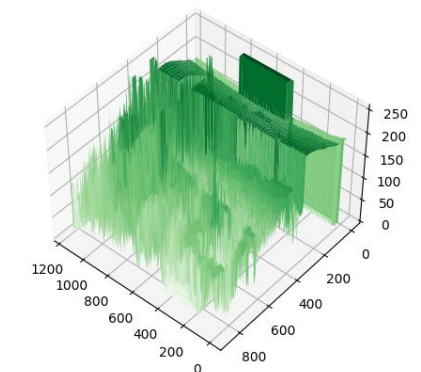
#RGB: 755844

- A 2D signal $f(x,y)$ obtained from a real measurable phenomenon



GENERATING 3D VIEWS

```
improc-0005.py > ...
1  # we import all libraries used in our program
2  import cv2
3  import matplotlib.pyplot as plt
4  from matplotlib import cm
5  import numpy as np
6  # we read an image from disk to memory
7  img = cv2.imread('img/jonkoping-01.jpg')
8  # we flip the image to get the good view during 3d projection
9  img2 = cv2.flip(src = img, flipCode = 1)
10 # we show the image and wait for a key to be pressed
11 cv2.imshow('Jonkoping', img)
12 key = cv2.waitKey(0)
13 # 3d mesh grid of an image
14 #source: https://www.geeksforgeeks.org/3d-surface-plotting-in-python-using-matplotlib/
15 # we make the data from our image
16 X = np.arange(img2.shape[1])
17 Y = np.arange(img2.shape[0])
18 X, Y = np.meshgrid(X, Y)
19 # we plot the surface
20 fig1, ax1 = plt.subplots(subplot_kw={"projection": "3d"})
21 fig2, ax2 = plt.subplots(subplot_kw={"projection": "3d"})
22 fig3, ax3 = plt.subplots(subplot_kw={"projection": "3d"})
23 ax1.plot_surface(X, Y, img2[:, :, 0], vmin = img2[:, :, 0].min() * 2, cmap = cm.Blues)
24 ax2.plot_surface(X, Y, img2[:, :, 1], vmin = img2[:, :, 1].min() * 2, cmap = cm.Greens)
25 ax3.plot_surface(X, Y, img2[:, :, 2], vmin = img2[:, :, 2].min() * 2, cmap = cm.Red)
26 # we show the surfaces
27 plt.show()
28 cv2.destroyAllWindows()
29
```

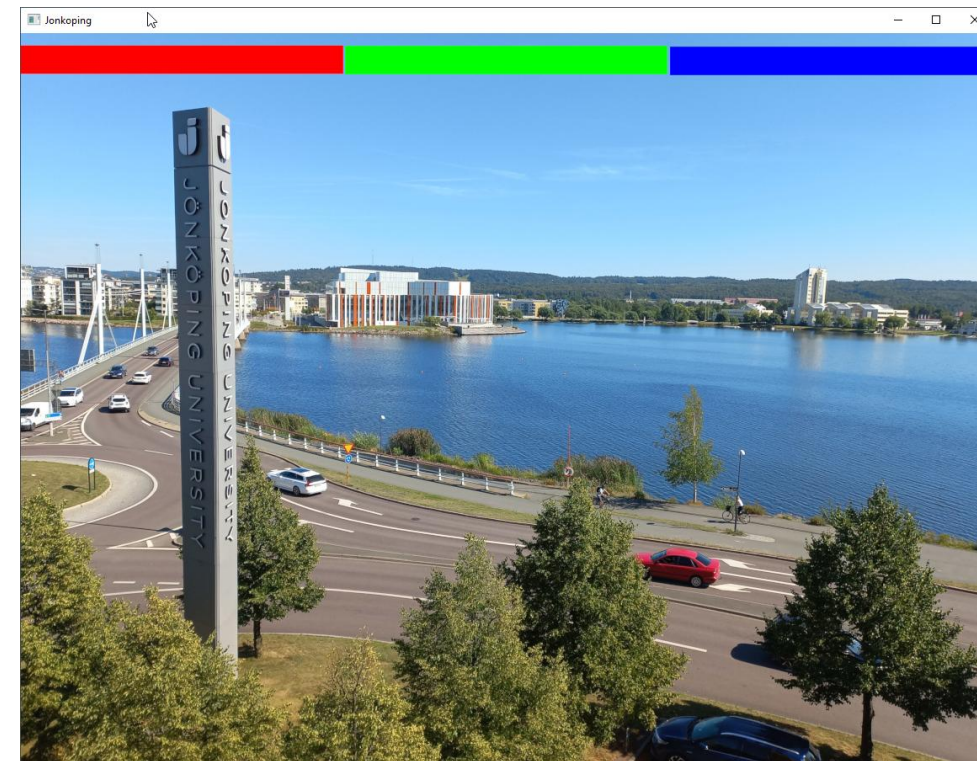


WHAT IS A DIGITAL IMAGE?

- A 2D array of pixels
 - Many algorithms available working on pixels, regions, sliding window, columns, rows:
 - Color spaces
 - Image resizing
 - Image thresholding
 - Image transformations: translation, rotation, scale, perspective projections...
 - Image segmentation
 - Sliding Window
 - ...
- A 2D signal $f(x,y)$ obtained from a real measurable phenomenon
 - Many algorithms coming from signal processing available:
 - Image filtering (convolution)
 - Edge detection
 - Fourier Transform
 - Wavelets Transforms
 - ...

READING AN IMAGE

```
improc-0000.py > ...
1  # we import all libraries used in our program
2  import cv2
3
4  # we read an image from disk to memory
5  img = cv2.imread('img/jonkoping-01.jpg')
6
7  # we print the information concerning this image
8  print('---> Shape: ', img.shape)
9  print('---> Size: ', img.size)
10 print('---> Number of dimensions', img.ndim)
11 print('---> Image nd-array: ', img)
12
13 # we show the image and wait for a key to be pressed
14 cv2.imshow('Jonkoping', img)
15 key = cv2.waitKey(0)
16
17 cv2.destroyAllWindows()
18
```



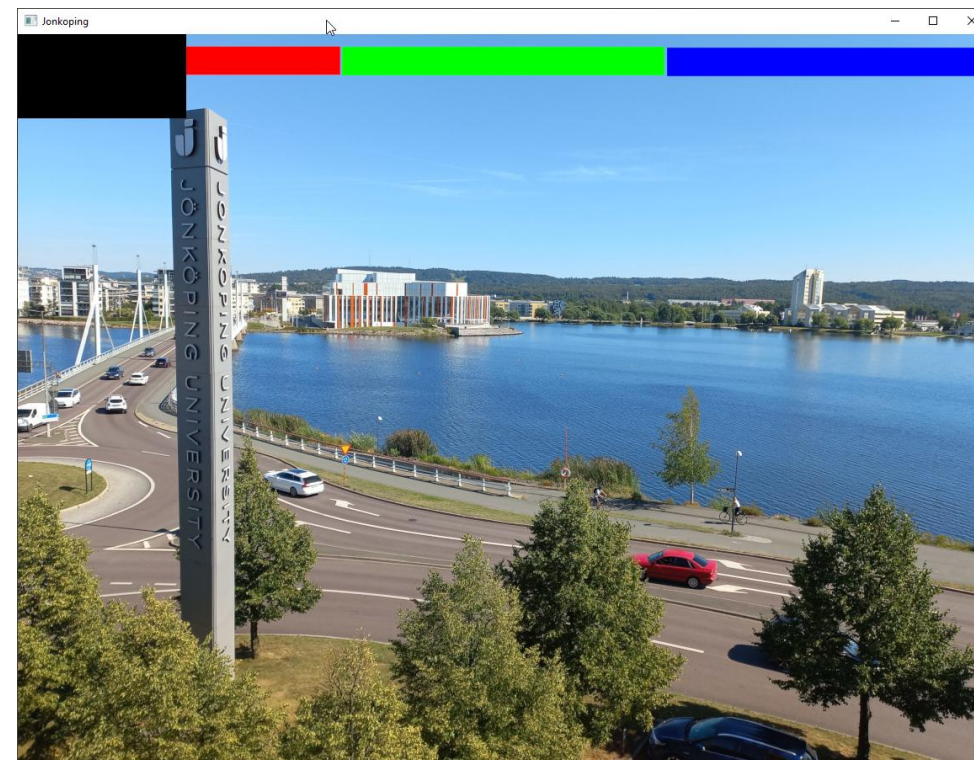
```
C:\Users\lanjer\Desktop\box\ju2023\teaching\courses\semester2\intelligent-mobile-platform\cours-jerome\code>python improc-0000.py
---> Shape: (867, 1156, 3)
---> Size: 3006756
---> Number of dimensions 3
---> Image nd-array: [[[244 183 119]
[244 183 119]
[244 181 120]
...
[247 189 124]
[247 189 124]
[247 189 124]]
[[[244 183 119]
[244 183 119]
[244 181 120]
...
[247 189 124]
```

Be careful: OpenCV uses the **BGR** pixel representation! For instance the first pixel of the image is [244 183 119], It is blue (upper left part of the sky) because of the high blue value: 244.

CHANGING THE PIXELS VALUES (1)

```
improc-0001.py > ...
1  # we import all libraries used in our program
2  import cv2
3
4  # we read an image from disk to memory
5  img = cv2.imread('img/jonkoping-01.jpg')
6
7  # we change some pixels values
8  img[0:100, 0:200, :] = 0
9
10 # we print the information concerning this image
11 print('Shape: ', img.shape)
12 print(img)
13
14 # we show the image and wait for a key to be pressed
15 cv2.imshow('Jonkoping', img)
16 key = cv2.waitKey(0)
17
18 cv2.destroyAllWindows()
19
```

```
PS C:\Users\lanjer\Desktop\ju2022\teaching\COURSES\semester2\intelligent-mobile-platform\cours-jerome\code> python .\improc-0001.py
Shape: (867, 1156, 3)
[[[ 0  0  0]
 [ 0  0  0]
 [ 0  0  0]
 ...
 [247 189 124]
 [247 189 124]
 [247 189 124]]
 [[ 0  0  0]
```



CHANGING THE PIXELS VALUES (2)

```
improc-0002.py > ...
1  # we import all libraries used in our program
2  import cv2
3
4  # we read an image from disk to memory
5  img = cv2.imread('img/jonkoping-01.jpg')
6
7  # we change some pixels values
8  img[:, :, 0] = 0
9
10 # we print the information concerning this image
11 print('Shape: ', img.shape)
12 print(img)
13
14 # we show the image and wait for a key to be pressed
15 cv2.imshow('Jonkoping', img)
16 key = cv2.waitKey(0)
17
18 cv2.destroyAllWindows()
19
```

```
PS C:\Users\lanjer\Desktop\ju2022\teaching\COURSES\semester2\intelligent-mobile-platform\cours-jerome\code> python .\improc-0002.py
Shape: (867, 1156, 3)
[[[ 0 183 119]
 [ 0 183 119]
 [ 0 181 120]
 ...
 [ 0 189 124]
 [ 0 189 124]
 [ 0 189 124]]
 [[ 0 183 119]
```



3. Methodology for Image Processing

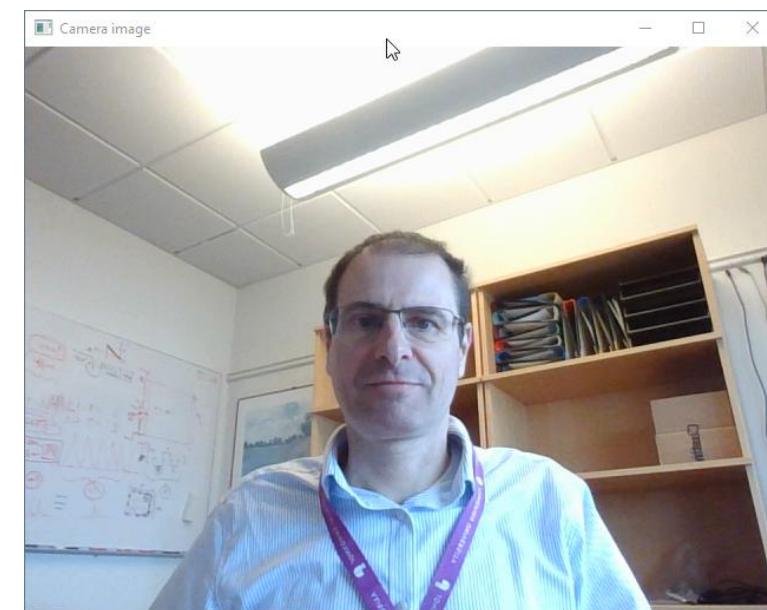
METHODOLOGY FOR IMAGE PROCESSING

- Where to start?
 - 1) How many image sources do I have/need?
 - 2) Is the camera is fixed or moving?
 - 3) What kind of images do I need?
 - 4) Do I need color images?
 - 5) Do I need to remove the background of my images?
 - 6) Do I need to find edges or regions in my images?
 - 7) Will I work on the whole image or on a region?
 - 8) What information do I want to extract from my images? How?
 - 9) Do I need Machine Learning to solve my problem?
 - 10) Can I find useful tools included in OpenCV?
 - ...

1) IMAGE SOURCES (WEBCAM)

- How many image sources do I have/need?
 - It depends on the application you want to build.
 - In general, we have an infinite loop to capture images from the camera and we process the images in the heart of this loop

```
improc-0003.py > ...
1  # we import all libraries used in our program
2  import cv2
3
4  # Initialize Camera
5  cap = cv2.VideoCapture(0)
6
7  # infinite loop
8  while True:
9      # we read an image from the camera
10     ret, frame = cap.read()
11     # there is an image
12     if ret:
13         frame = cv2.flip(frame, flipCode = 1)
14         # we display it
15         cv2.imshow('Camera image', frame)
16         # we wait for a key pressed
17         k = cv2.waitKey(10)
18         # if it is 'q', we quit
19         if k == ord('q'):
20             break
21     # there is no image
22     else:
23         print('ERROR ---> Camera not found!')
24
25 # we close all windows
26 cv2.destroyAllWindows()
27
```



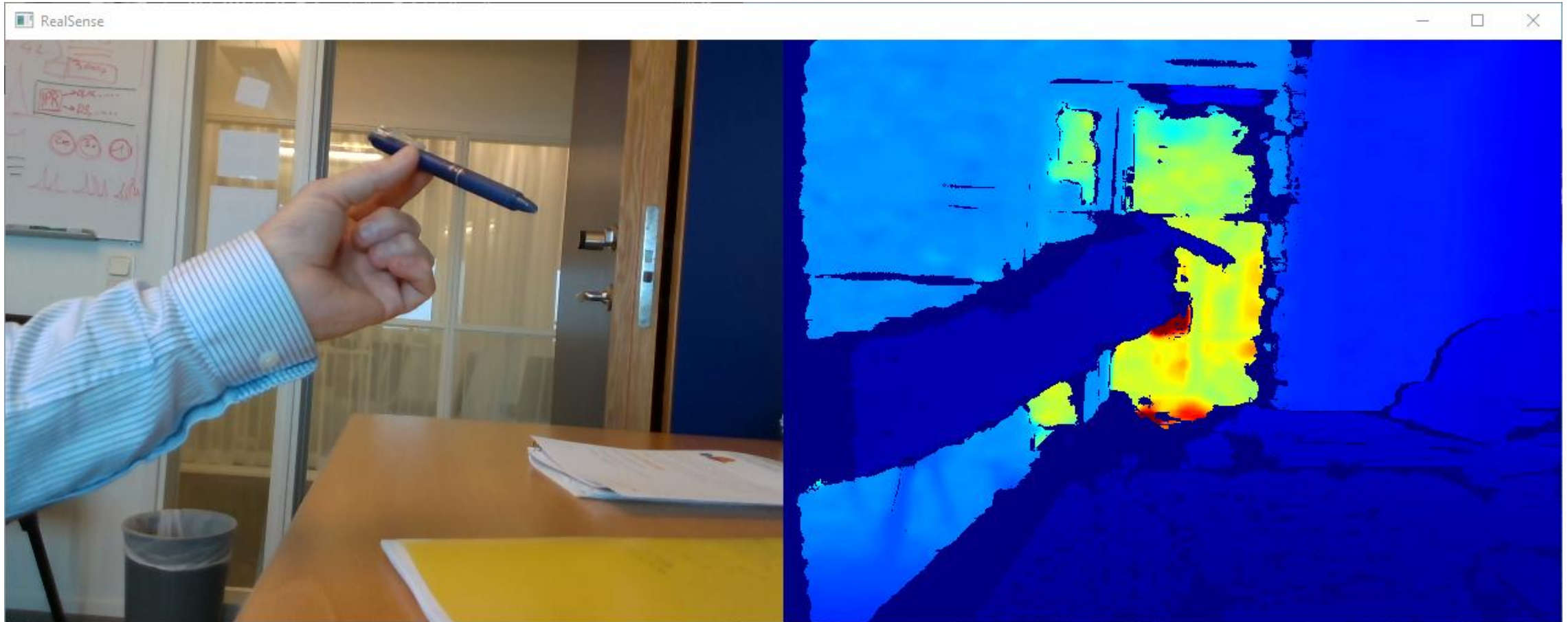
1) IMAGE SOURCES (INTEL REALSENSE)

```
improc-0020.py > ...
1  import pyrealsense2 as rs
2  import numpy as np
3  import cv2
4  # Configure depth and color streams
5  pipeline = rs.pipeline()
6  config = rs.config()
7  # Get device product line for setting a supporting resolution
8  pipeline_wrapper = rs.pipeline_wrapper(pipeline)
9  pipeline_profile = config.resolve(pipeline_wrapper)
10 device = pipeline_profile.get_device()
11 device_product_line = str(device.get_info(rs.camera_info.product_line))
12
13 found_rgb = False
14 for s in device.sensors:
15     if s.get_info(rs.camera_info.name) == 'RGB Camera':
16         found_rgb = True
17         break
18 if not found_rgb:
19     print("The demo requires Depth camera with Color sensor")
20     exit(0)
21
22 config.enable_stream(rs.stream.depth, 640, 480, rs.format.z16, 30)
23 if device_product_line == 'L500':
24     config.enable_stream(rs.stream.color, 960, 540, rs.format.bgr8, 30)
25 else:
26     config.enable_stream(rs.stream.color, 640, 480, rs.format.bgr8, 30)
27 # Start streaming
28 pipeline.start(config)
```

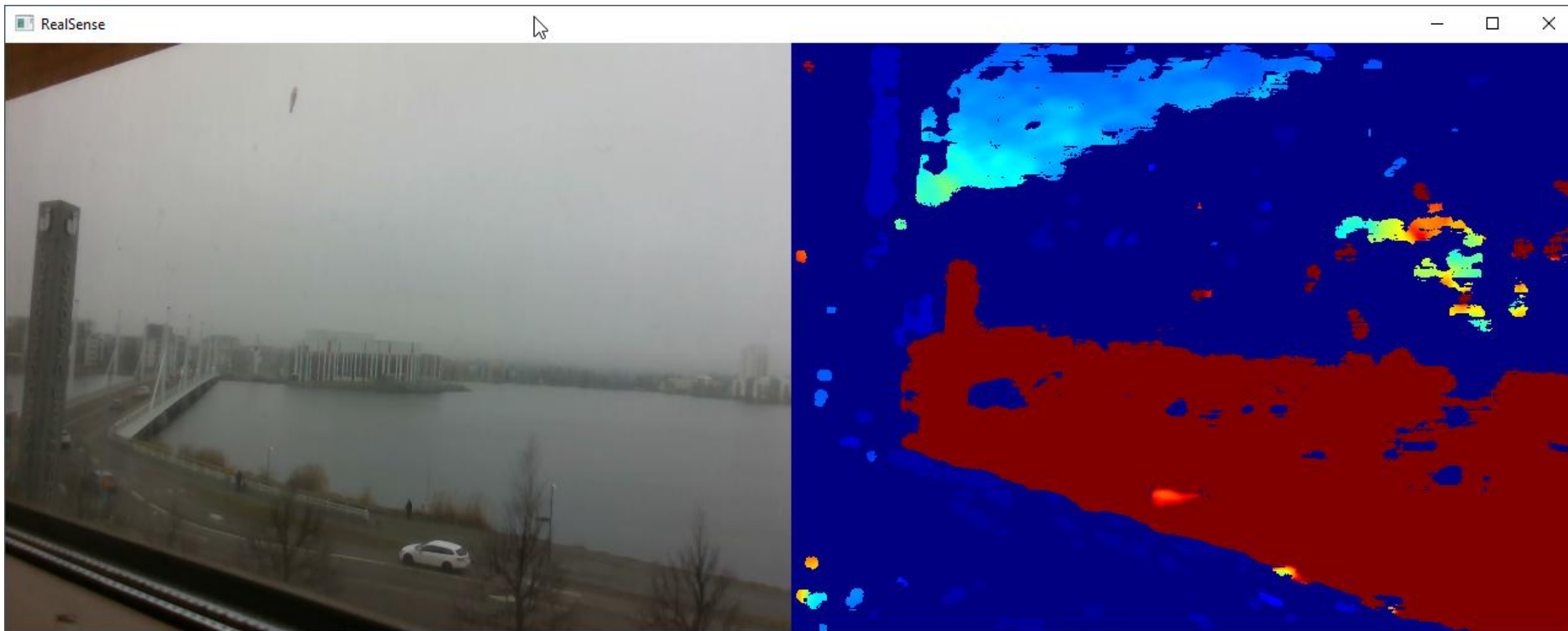
```
29 try:
30     while True:
31         # Wait for a coherent pair of frames: depth and color
32         frames = pipeline.wait_for_frames()
33         depth_frame = frames.get_depth_frame()
34         color_frame = frames.get_color_frame()
35         if not depth_frame or not color_frame:
36             continue
37
38         # Convert images to numpy arrays
39         depth_image = np.asanyarray(depth_frame.get_data())
40         color_image = np.asanyarray(color_frame.get_data())
41
42         # Apply colormap on depth image (image must be converted to 8-bit per pixel first)
43         depth_colormap = cv2.applyColorMap(cv2.convertScaleAbs(depth_image, alpha=0.03), cv2.COLORMAP_JET)
44
45         depth_colormap_dim = depth_colormap.shape
46         color_colormap_dim = color_image.shape
47
48         # If depth and color resolutions are different, resize color image to match depth image for display
49         if depth_colormap_dim != color_colormap_dim:
50             resized_color_image = cv2.resize(color_image, dsize=(depth_colormap_dim[1],
51                                                                     depth_colormap_dim[0]), interpolation=cv2.INTER_AREA)
52             images = np.hstack((resized_color_image, depth_colormap))
53         else:
54             images = np.hstack((color_image, depth_colormap))
55         # Show images
56         cv2.namedWindow('RealSense', cv2.WINDOW_AUTOSIZE)
57         cv2.imshow('RealSense', images)
58         k = cv2.waitKey(1)
59         if k==ord('q'):
60             break
61 finally:
62     # Stop streaming
63     pipeline.stop()
64     cv2.destroyAllWindows()
65
```

Be careful: the code works with an Intel Realsense camera only,
If you have another camera, you must change this code.

1) IMAGE SOURCES (INTEL REALSENSE)



1) IMAGE SOURCES (INTEL REALSENSE)



1) IMAGE SOURCES (BOTH)

```
improc-0030.py > ...
1  import pyrealsense2 as rs
2  import numpy as np
3  import cv2
4  # Configure depth and color streams
5  pipeline = rs.pipeline()
6  config = rs.config()
7  # Get device product line for setting a supporting resolution
8  pipeline_wrapper = rs.pipeline_wrapper(pipeline)
9  pipeline_profile = config.resolve(pipeline_wrapper)
10 device = pipeline_profile.get_device()
11 device_product_line = str(device.get_info(rs.camera_info.product_line))
12
13 found_rgb = False
14 for s in device.sensors:
15     if s.get_info(rs.camera_info.name) == 'RGB Camera':
16         found_rgb = True
17         break
18 if not found_rgb:
19     print("The demo requires Depth camera with Color sensor")
20     exit(0)
21
22 config.enable_stream(rs.stream.depth, 640, 480, rs.format.z16, 30)
23 if device_product_line == 'L500':
24     config.enable_stream(rs.stream.color, 960, 540, rs.format.bgr8, 30)
25 else:
26     config.enable_stream(rs.stream.color, 640, 480, rs.format.bgr8, 30)
27 # Start streaming
28 pipeline.start(config)
29
30 # Initialize Camera
31 cap = cv2.VideoCapture(0)
32 try:
33     while True:
```

```
34
35     # we read an image from the camera
36     ret, frame = cap.read()
37     # there is an image
38     if ret:
39         frame = cv2.flip(frame, flipCode = 1)
40         # we display it
41         cv2.imshow('Camera image', frame)
42     # Wait for a coherent pair of frames: depth and color
43     frames = pipeline.wait_for_frames()
44     depth_frame = frames.get_depth_frame()
45     color_frame = frames.get_color_frame()
46     if not depth_frame or not color_frame:
47         continue
48     # Convert images to numpy arrays
49     depth_image = np.asanyarray(depth_frame.get_data())
50     depth_image = cv2.flip(depth_image, flipCode = 1)
51     color_image = np.asanyarray(color_frame.get_data())
52     color_image = cv2.flip(color_image, flipCode = 1)
53
54     # Apply colormap on depth image (image must be converted to 8-bit per pixel first)
55     depth_colormap = cv2.applyColorMap(cv2.convertScaleAbs(depth_image, alpha=0.03), cv2.COLORMAP_JET)
56
57     depth_colormap_dim = depth_colormap.shape
58     color_colormap_dim = color_image.shape
59
60     # If depth and color resolutions are different, resize color image to match depth image for display
61     if depth_colormap_dim != color_colormap_dim:
62         resized_color_image = cv2.resize(color_image, dsize=(depth_colormap_dim[1],
63                                                             depth_colormap_dim[0]), interpolation=cv2.INTER_AREA)
64         images = np.hstack((resized_color_image, depth_colormap))
65     else:
66         images = np.hstack((color_image, depth_colormap))
67     # Show images
68     cv2.namedWindow('RealSense', cv2.WINDOW_AUTOSIZE)
69     cv2.imshow('RealSense', images)
70     k = cv2.waitKey(10)
71     if k==ord('q'):
72         break
73 finally:
74     # Stop streaming
75     pipeline.stop()
76     cv2.destroyAllWindows()
```



2) FIXED OR MOVING CAMERA

- The problem is not exactly the same with a fixed or a moving camera
- **But** there is always **NOISE** on your images in both case
- Let's do an **experiment**

```
improc-0031.py > [🔗] depth_colormap_dim
29 pipeline.start(config)
30
31 # Initialize Camera
32 cap = cv2.VideoCapture(0)
33 ret, frame = cap.read()
34 x = datetime.datetime.now()
35 fileRGB = 'outRGB'+str(x.year)+str(x.month)+str(x.day)+str(x.hour)+str(x.minute)+str(x.second)+'.avi'
36 outRGB = cv2.VideoWriter(fileRGB,cv2.VideoWriter_fourcc(*'MJPG'), 30, (frame.shape[1],frame.shape[0]))
37 numberOfImagesInVideo = 300
38 nb = 0
39
40 try:
41     while True:
42         # we read an image from the camera
43         ret, frame = cap.read()
44         # there is an image
45         if ret:
46             frame = cv2.flip(frame, flipCode = 1)
47             if nb<numberOfImagesInVideo:
48                 outRGB.write(frame)
49                 nb += 1
50             # we display it
51             cv2.imshow('Camera image', frame)
52             # Wait for a coherent pair of frames: depth and color
53             frames = pipeline.wait_for_frames()
54             depth_frame = frames.get_depth_frame()
```

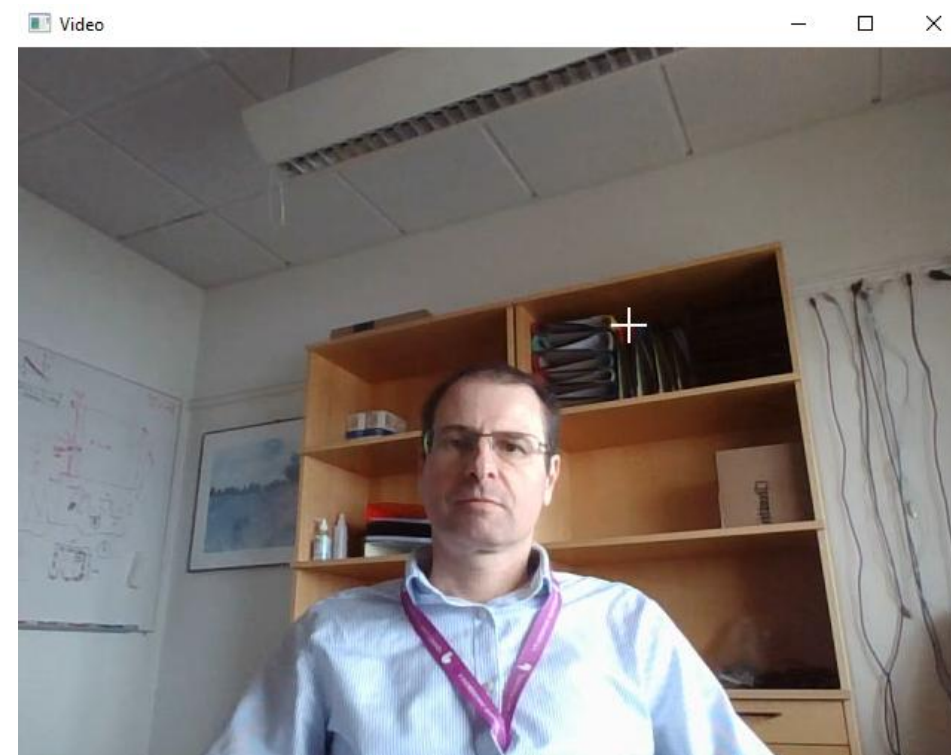
2) FIXED OR MOVING CAMERA

- We obtain a **video file**
- Let's try to see if the image pixels are "stable" in time!



2) FIXED OR MOVING CAMERA

```
improc-0032.py > ...
1  import cv2
2  import numpy as np
3
4  #source: https://learnopencv.com/read-write-and-display-a-video-using-opencv-cpp-python/
5  cap = cv2.VideoCapture('outRGB2023414131958.avi')
6
7  # Check if camera opened successfully
8  if (cap.isOpened() == False):
9      print("Error opening video stream or file")
10
11  # Read until video is completed
12  while(cap.isOpened()):
13      # Capture frame-by-frame
14      ret, frame = cap.read()
15      if ret == True:
16          # Display the resulting frame
17          cv2.imshow('Video', frame)
18          # Press Q on keyboard to exit
19          k = cv2.waitKey(25)
20          if k == ord('q'):
21              break
22      else:
23          break
24
25  # When everything done, release the video capture object
26  cap.release()
27
28  # Closes all the frames
29  cv2.destroyAllWindows()
30
```



2) FIXED OR MOVING CAMERA

```
improc-0034.py > ...
1  import cv2
2  import datetime
3
4  #source: https://learnopencv.com/read-write-and-display-a-video-using-opencv-cpp-python/
5  cap = cv2.VideoCapture('outRGB2023414131958.avi')
6
7  # Check if camera opened successfully
8  if (cap.isOpened() == False):
9      print("Error opening video stream or file")
10
11  x = datetime.datetime.now()
12  ret, frame = cap.read()
13  fileRGB = 'noiseRGB'+str(x.year)+str(x.month)+str(x.day)+str(x.hour)+str(x.minute)+str(x.second)+'.avi'
14  outRGB = cv2.VideoWriter(fileRGB, cv2.VideoWriter_fourcc(*'MJPG'), 30, (frame.shape[1], frame.shape[0]))
15
16  ret, oldFrame = cap.read()
17
18  # Read until video is completed
19  while(cap.isOpened()):
20      # Capture frame-by-frame
21      ret, newFrame = cap.read()
22      if ret == True:
23          # Display the resulting frame
24          cv2.imshow('Video', newFrame-oldFrame)
25          outRGB.write(newFrame-oldFrame)
26
27          oldFrame = newFrame.copy()
28          # Press Q on keyboard to exit
29          k = cv2.waitKey(25)
30          if k == ord('q'):
31              break
32      else:
33          break
34
35  # When everything done, release the video capture object
36  cap.release()
37  outRGB.release()
38
39  # Closes all the frames
40  cv2.destroyAllWindows()
41
```



2) FIXED OR MOVING CAMERA

```
improc-0035.py > ...
1  import cv2
2  import datetime
3
4  #source: https://learnopencv.com/read-write-and-display-a-video-using-opencv-cpp-python/
5  cap = cv2.VideoCapture('outRGB2023414131958.avi')
6
7  # Check if camera opened successfully
8  if (cap.isOpened() == False):
9      print("Error opening video stream or file")
10
11  x = datetime.datetime.now()
12  ret, frame = cap.read()
13  fileRGB = 'noiseRGB'+str(x.year)+str(x.month)+str(x.day)+str(x.hour)+str(x.minute)+str(x.second)+'.avi'
14  outRGB = cv2.VideoWriter(fileRGB, cv2.VideoWriter_fourcc(*'MJPG'), 30, (frame.shape[1], frame.shape[0]))
15
16  ret, oldFrame = cap.read()
17  oldFrame = cv2.blur(oldFrame, (5, 5))
18
19  # Read until video is completed
20  while(cap.isOpened()):
21      # Capture frame-by-frame
22      ret, newFrame = cap.read()
23
24      if ret == True:
25          # Display the resulting frame
26          newFrame = cv2.blur(newFrame, (5, 5))
27          cv2.imshow('Video', newFrame-oldFrame)
28          outRGB.write(newFrame-oldFrame)
29
30          oldFrame = newFrame.copy()
31          # Press Q on keyboard to exit
32          k = cv2.waitKey(25)
33          if k == ord('q'):
34              break
35      else:
36          break
37
38  # When everything done, release the video capture object
39  cap.release()
40  outRGB.release()
41
42  # Closes all the frames
43  cv2.destroyAllWindows()
44
```

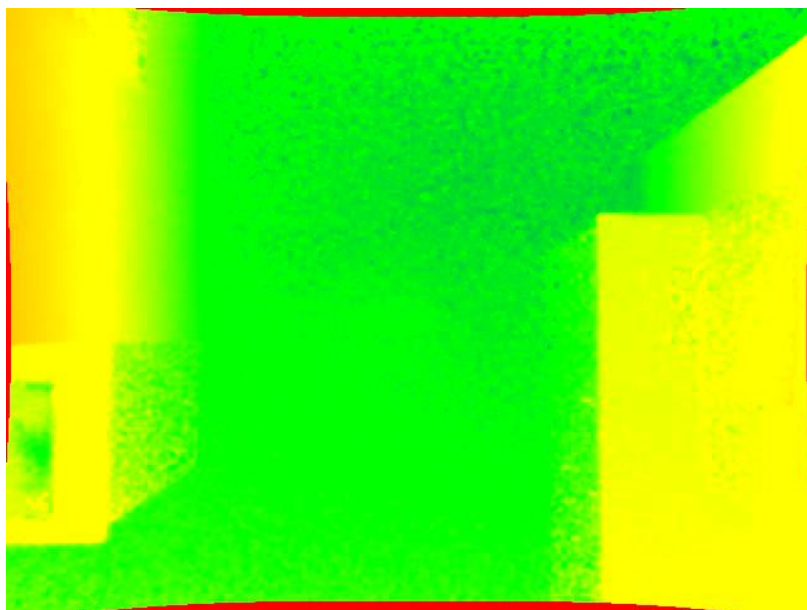
- GaussianBlur and medianBlur can also help...



3) TYPES OF IMAGES

- Depending on your camera sensors and needs, you can use many types of cameras:

DEPTH



INFRARED



RGB



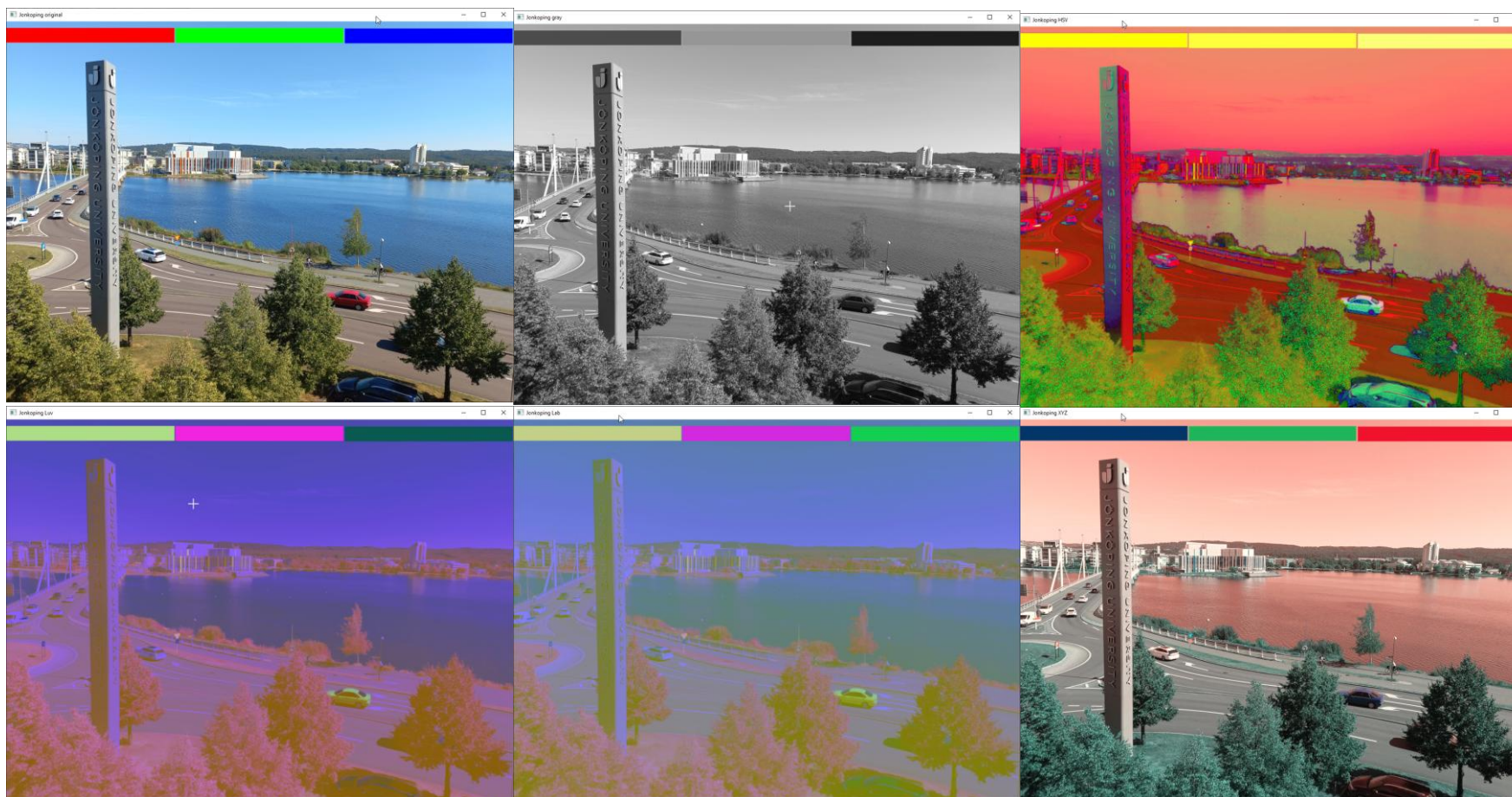
LIDAR, MEDICAL, MULTISPECTRAL, AERIAL, SATELITE IMAGES...

4) COLOR IMAGES OR NOT...

- RGB, Grayscale, HSV, HSL, XYZ, Lab, Luv... colorspace

```
improc-0040.py > ...
1  # we import all libraries used in our program
2  import cv2
3  import numpy
4
5  # we read an image from disk to memory
6  img = cv2.imread('img/jonkoping-01.jpg')
7  imgGray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
8  imgHSV = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
9  imgLab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
10 imgLuv = cv2.cvtColor(img, cv2.COLOR_BGR2LUV)
11 imgXYZ = cv2.cvtColor(img, cv2.COLOR_BGR2XYZ)
12 print("---> Stats:")
13 print("shape, (min, max, avg) RGB: ", img.shape, ', ', numpy.min(img), numpy.max(img), numpy.mean(img))
14 print("shape, (min, max, avg) Gray: ", imgGray.shape, ', ', numpy.min(imgGray), numpy.max(imgGray), numpy.mean(imgGray))
15 print("shape, (min, max, avg) HSV: ", imgHSV.shape, ', ', numpy.min(imgHSV), numpy.max(imgHSV), numpy.mean(imgHSV))
16 print("shape, (min, max, avg) Lab: ", imgLab.shape, ', ', numpy.min(imgLab), numpy.max(imgLab), numpy.mean(imgLab))
17 print("shape, (min, max, avg) Luv: ", imgLuv.shape, ', ', numpy.min(imgLuv), numpy.max(imgLuv), numpy.mean(imgLuv))
18 print("shape, (min, max, avg) XYZ: ", imgXYZ.shape, ', ', numpy.min(imgXYZ), numpy.max(imgXYZ), numpy.mean(imgXYZ))
19 # we show the image and wait for a key to be pressed
20 cv2.imshow('Jonkoping original', img)
21 cv2.imshow('Jonkoping gray', imgGray)
22 cv2.imshow('Jonkoping Lab', imgLab)
23 cv2.imshow('Jonkoping HSV', imgHSV)
24 cv2.imshow('Jonkoping Luv', imgLuv)
25 cv2.imshow('Jonkoping XYZ', imgXYZ)
26 key = cv2.waitKey(0)
27
28 cv2.destroyAllWindows()
29
```

4) COLOR IMAGES OR NOT...



```
PS C:\Users\lanjer\Desktop\ju2022\teaching\COURSES\semester2\intelligent-mobile-platform\cours-jerome\code> python .\improc-0040.py
--> Stats:
shape, (min, max, avg) RGB: (867, 1156, 3) , 0 255 133.69111128405498
shape, (min, max, avg) Gray: (867, 1156) , 4 255 132.29723762087778
shape, (min, max, avg) HSV: (867, 1156, 3) , 0 255 114.5795405413675
shape, (min, max, avg) Lab: (867, 1156, 3) , 2 255 130.53138565284314
shape, (min, max, avg) Luv: (867, 1156, 3) , 3 255 119.25149164082487
shape, (min, max, avg) XYZ: (867, 1156, 3) , 1 255 137.87846236941076
```


5) BACKGROUND REMOVAL

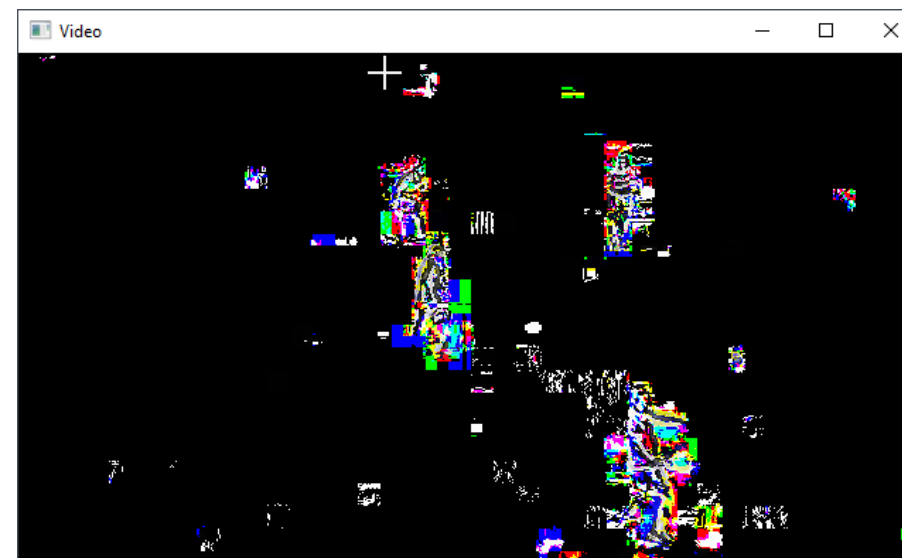
- In order to study the moving parts in a video (here the players and the ball), we can try to guess the background of the image and see what have moved by a simple difference (bgImg-CurrentImg)
- How to get the background of a video?



Source video: <https://www.youtube.com/watch?v=P6yfa7rfc6o>
YT2MP4 Converter: <https://wave.video/convert/youtube-to-mp4-85>

5) BACKGROUND REMOVAL

```
improc-0050.py > ...
1  import cv2
2  import numpy as np
3
4  #source: https://learnopencv.com/read-write-and-display-a-video-using-opencv-cpp-python/
5  cap = cv2.VideoCapture('padelpoint.mp4')
6  # Check if camera opened successfully
7  if (cap.isOpened() == False):
8      print("Error opening video stream or file")
9  ret, oldFrame = cap.read()
10 # Read until video is completed
11 while(cap.isOpened()):
12     # Capture frame-by-frame
13     ret, newFrame = cap.read()
14     if ret == True:
15         # Display the resulting frame
16         cv2.imshow('Video', oldFrame-newFrame)
17         oldFrame = newFrame.copy()
18         # Press Q on keyboard to exit
19         k = cv2.waitKey(25)
20         if k == ord('q'):
21             break
22     else:
23         break
24
25 # When everything done, release the video capture object
26 cap.release()
27 # Closes all the frames
28 cv2.destroyAllWindows()
29
```



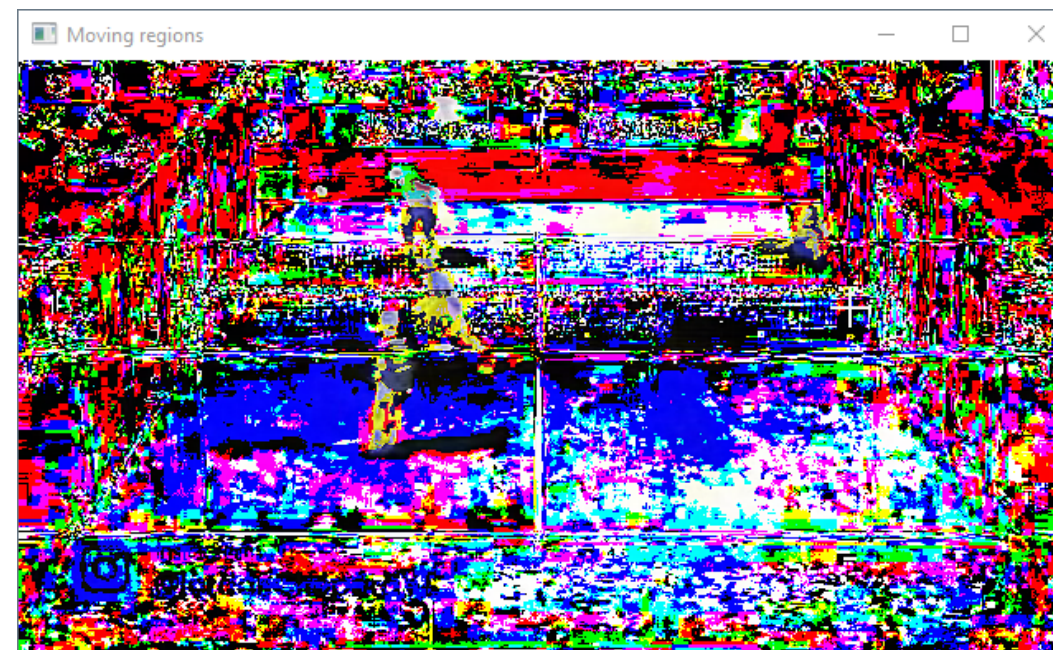
5) BACKGROUND REMOVAL (1ST TRY)

```
improc-0051.py > ...
1  import cv2
2  import numpy
3
4  #source: https://learnopencv.com/read-write-and-display-a-video-using-opencv-cpp-python/
5  cap = cv2.VideoCapture('padelpoint.mp4')
6  # Check if camera opened successfully
7  if (cap.isOpened() == False):
8      print("Error opening video stream or file")
9  ret, frame = cap.read()
10  sumImages = numpy.zeros(frame.shape)
11  nb = 0
12
13  # Read until video is completed
14  while(cap.isOpened()):
15      # Capture frame-by-frame
16      ret, frame = cap.read()
17      if ret == True:
18          # add the image to the sum
19          sumImages += frame
20          nb += 1
21      else:
22          break
23
24  avgImages = sumImages / nb
25  avgImages = avgImages.astype(numpy.uint8)
26  cv2.imshow('sumImages', avgImages)
27  key = cv2.waitKey(0)
28
29  # When everything done, release the video capture object
30  cap.release()
31  # Closes all the frames
32  cv2.destroyAllWindows()
33
```



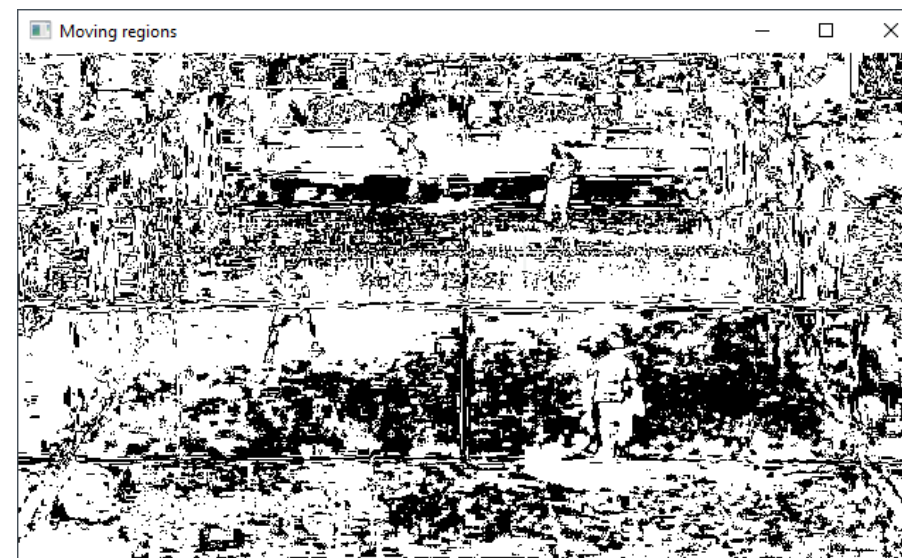
5) BACKGROUND REMOVAL (2ND TRY)

```
improc-0052.py > ...
18     # add the image to the sum
19     sumImages += frame
20     nb += 1
21 else:
22     break
23
24 avgImages = sumImages / nb
25 avgImages = avgImages.astype(numpy.uint8)
26 cv2.imshow('sumImages', avgImages)
27 key = cv2.waitKey(0)
28
29 backgroundImage = avgImages.copy()
30 # When everything done, release the video capture object
31 cap.release()
32
33 cap = cv2.VideoCapture('padelpoint.mp4')
34 # Check if camera opened successfully
35 if (cap.isOpened() == False):
36     print("Error opening video stream or file")
37 ret, frame = cap.read()
38 sumImages = numpy.zeros(frame.shape)
39 nb = 0
40
41 # Read until video is completed
42 while(cap.isOpened()):
43     # Capture frame-by-frame
44     ret, frame = cap.read()
45     cv2.imshow('Moving regions', backgroundImage-frame)
46     key = cv2.waitKey(10)
47     if key == ord('q'):
48         break
49
50 # When everything done, release the video capture object
51 cap.release()
52 # Closes all the frames
53 cv2.destroyAllWindows()
54
```



5) BACKGROUND REMOVAL (3RD TRY)

```
improc-0053.py > ...
20     sumImages += cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
21     nb += 1
22 else:
23     break
24
25 avgImages = sumImages / nb
26 avgImages = avgImages.astype(numpy.uint8)
27 cv2.imshow('sumImages', avgImages)
28 key = cv2.waitKey(0)
29
30 backgroundImage = avgImages.copy()
31 # When everything done, release the video capture object
32 cap.release()
33
34 cap = cv2.VideoCapture('padelpoint.mp4')
35 # Check if camera opened successfully
36 if (cap.isOpened() == False):
37     print("Error opening video stream or file")
38 ret, frame = cap.read()
39 sumImages = numpy.zeros(frame.shape)
40 nb = 0
41
42 # Read until video is completed
43 while(cap.isOpened()):
44     # Capture frame-by-frame
45     ret, frame = cap.read()
46     diff = (((backgroundImage-cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY))<200)*255).astype(numpy.uint8)
47     print(diff)
48     cv2.imshow('Moving regions', diff)
49     key = cv2.waitKey(10)
50     if key == ord('q'):
51         break
52
53 # When everything done, release the video capture object
54 cap.release()
55 # Closes all the frames
56 cv2.destroyAllWindows()
57
```

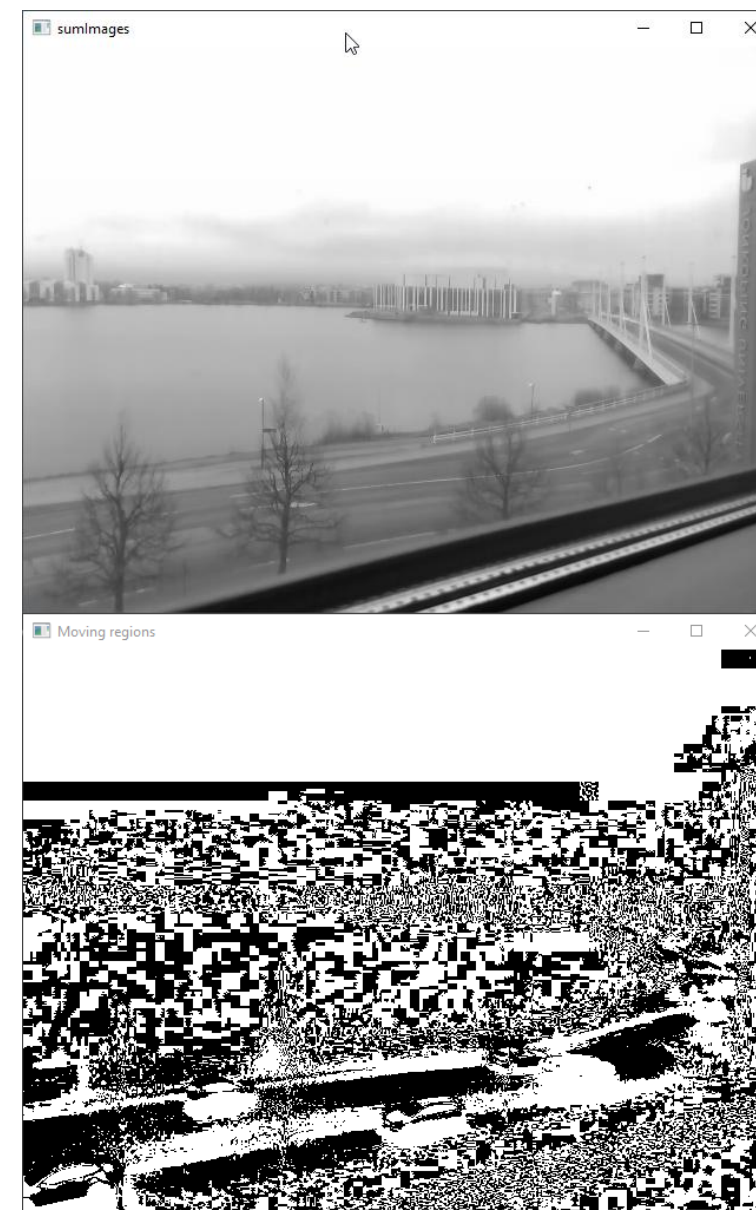


5) BACKGROUND REMOVAL



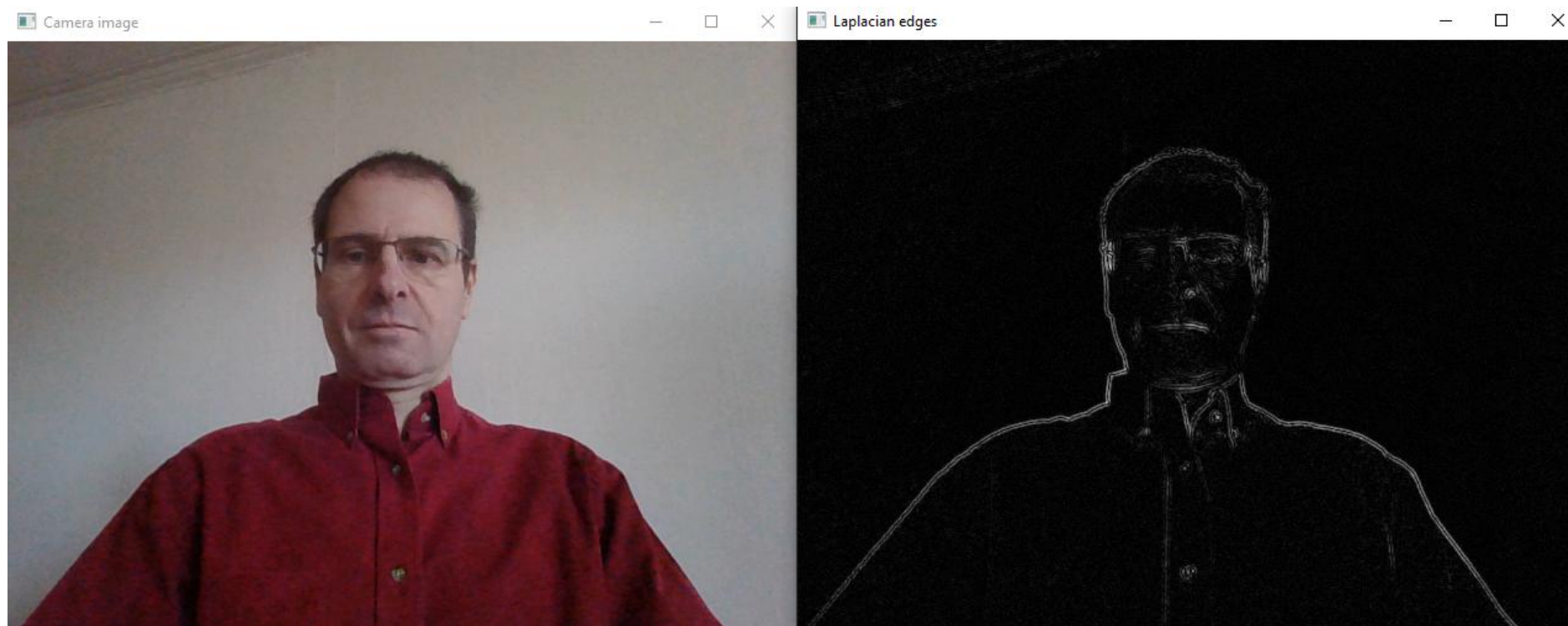
5) BACKGROUND REMOVAL

```
improc-0054.py > ...
23     break
24
25     avgImages = sumImages / nb
26     avgImages = avgImages.astype(numpy.uint8)
27     cv2.imshow('sumImages', avgImages)
28     key = cv2.waitKey(0)
29
30     backgroundImage = avgImages.copy()
31     # When everything done, release the video capture object
32     cap.release()
33
34     cap = cv2.VideoCapture('bridgeRGB2023414161111.avi')
35     # Check if camera opened successfully
36     if (cap.isOpened() == False):
37         print("Error opening video stream or file")
38     ret, frame = cap.read()
39     sumImages = numpy.zeros(frame.shape)
40     nb = 0
41
42     # Read until video is completed
43     while(cap.isOpened()):
44         # Capture frame-by-frame
45         ret, frame = cap.read()
46         diff = (((backgroundImage - cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)) < 200) * 255).astype(numpy.uint8)
47         print(diff)
48         cv2.imshow('Moving regions', diff)
49         key = cv2.waitKey(10)
50         if key == ord('q'):
51             break
52
53     # When everything done, release the video capture object
54     cap.release()
55     # Closes all the frames
56     cv2.destroyAllWindows()
57
```



6) EDGES AND REGIONS

- In many applications of Image Processing, you need to find **edges** and **regions** in an image.



6) EDGES AND REGIONS

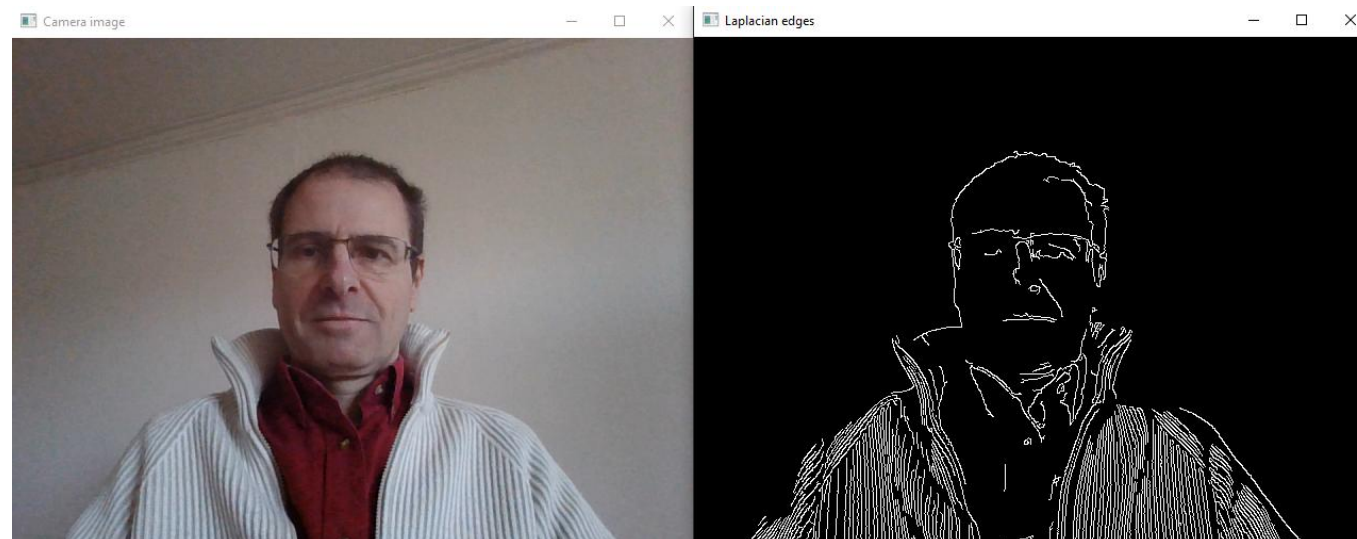
```
improc-0060.py > ...
1  # we import all libraries used in our program
2  import cv2
3
4  # Initialize Camera
5  cap = cv2.VideoCapture(0)
6
7  # infinite loop
8  while True:
9      # we read an image from the camera
10     ret, frame = cap.read()
11     # there is an image
12     if ret:
13         frame = cv2.flip(frame, flipCode = 1)
14         frameBlur = cv2.GaussianBlur(frame, (3, 3), 0)
15         frameGray = cv2.cvtColor(frameBlur, cv2.COLOR_BGR2GRAY)
16         frameEdges = cv2.Laplacian(frameGray, cv2.CV_16S, ksize=3)
17         frameFinalEdges = cv2.convertScaleAbs(frameEdges)
18         # we display it
19         cv2.imshow('Camera image', frame)
20         cv2.imshow('Laplacian edges', frameFinalEdges)
21         # we wait for a key pressed
22         k = cv2.waitKey(10)
23         # if it is 'q', we quit
24         if k == ord('q'):
25             break
26     # there is no image
27     else:
28         print('ERROR ---> Camera not found!')
29
30 # we close all windows
31 cv2.destroyAllWindows()
```

Laplacian filter



6) EDGES AND REGIONS

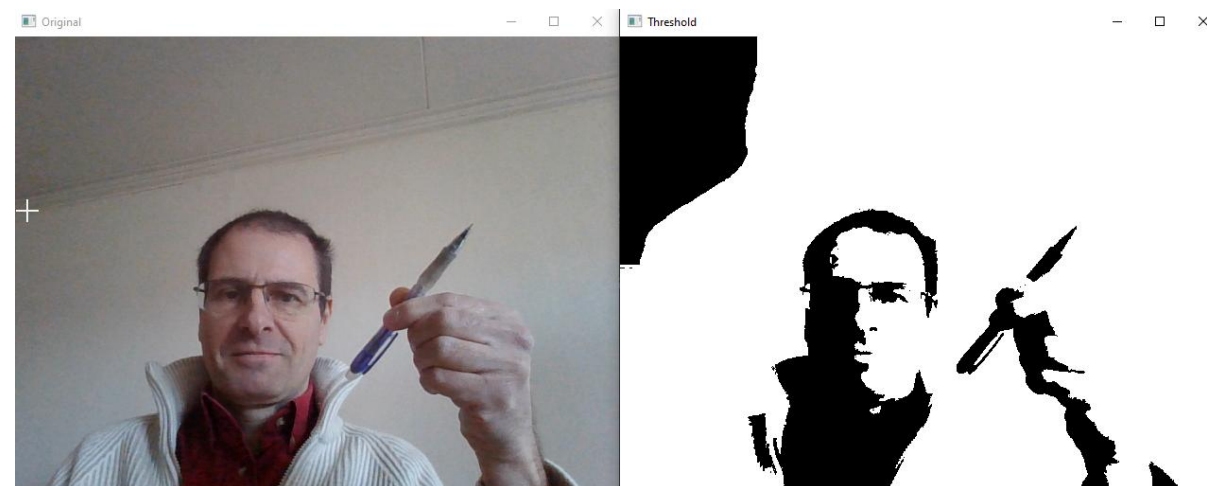
```
improc-0061.py > ...
1  # we import all libraries used in our program
2  import cv2
3
4  # Initialize Camera
5  cap = cv2.VideoCapture(0)
6
7  # infinite loop
8  while True:
9      # we read an image from the camera
10     ret, frame = cap.read()
11     # there is an image
12     if ret:
13         frame = cv2.flip(frame, flipCode = 1)
14         frameEdges = cv2.Canny(frame, 100, 200)
15         # we display it
16         cv2.imshow('Camera image', frame)
17         cv2.imshow('Laplacian edges', frameEdges)
18         # we wait for a key pressed
19         k = cv2.waitKey(10)
20         # if it is 'q', we quit
21         if k == ord('q'):
22             break
23     # there is no image
24     else:
25         print('ERROR ---> Camera not found!')
26
27 # we close all windows
28 cv2.destroyAllWindows()
29
```



Canny Edge Detector

6) EDGES AND REGIONS

```
improc-0062.py > ...
1  # we import all libraries used in our program
2  import cv2
3  # Initialize Camera
4  cap = cv2.VideoCapture(0)
5  # infinite loop
6  while True:
7      # we read an image from the camera
8      ret, frame = cap.read()
9      # there is an image
10     if ret:
11         frame = cv2.flip(frame, flipCode = 1)
12         #Edge-based segmentation
13         #source: https://pyimagesearch.com/2015/11/02/watershed-opencv/
14         shifted = cv2.pyrMeanShiftFiltering(frame, 21, 51)
15         # convert the mean shift image to grayscale, then apply
16         # Otsu's thresholding
17         gray = cv2.cvtColor(shifted, cv2.COLOR_BGR2GRAY)
18         thresh = cv2.threshold(gray, 0, 255,
19                               | cv2.THRESH_BINARY | cv2.THRESH_OTSU)[1]
20         # we display it
21         cv2.imshow("Original", frame)
22         cv2.imshow("Threshold", thresh)
23
24         # we wait for a key pressed
25         k = cv2.waitKey(10)
26         # if it is 'q', we quit
27         if k == ord('q'):
28             break
29
30     # we close all windows
31     cv2.destroyAllWindows()
32
```



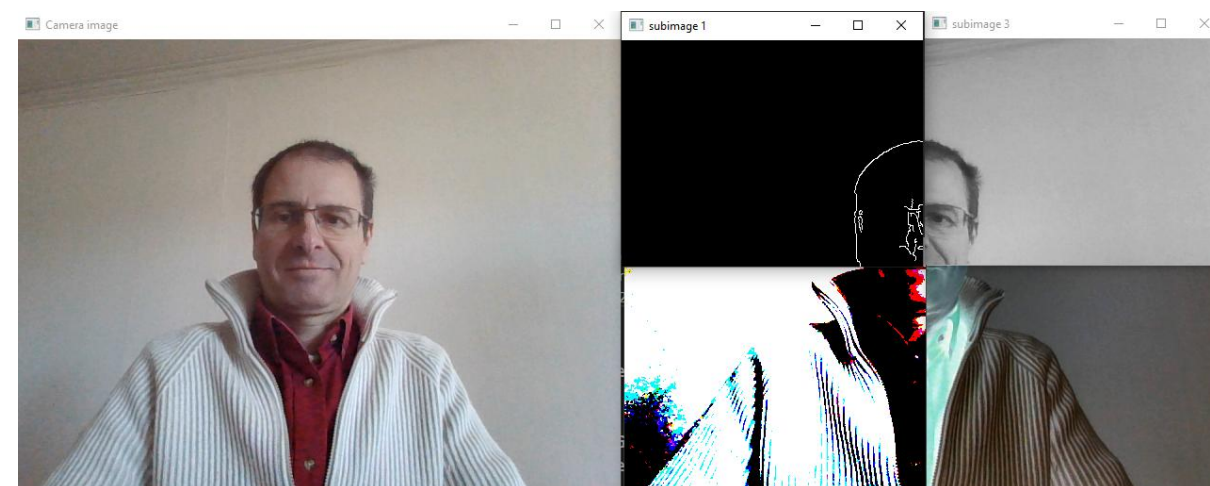
Watershed algorithm

7) REGION OF INTEREST

- When working on an image, we can choose to work only in a sub-image:
 - The interest thing happens only in a small part of the image so it is unnecessary to work on the whole image
 - It saves computing time to work only on a small part of the image

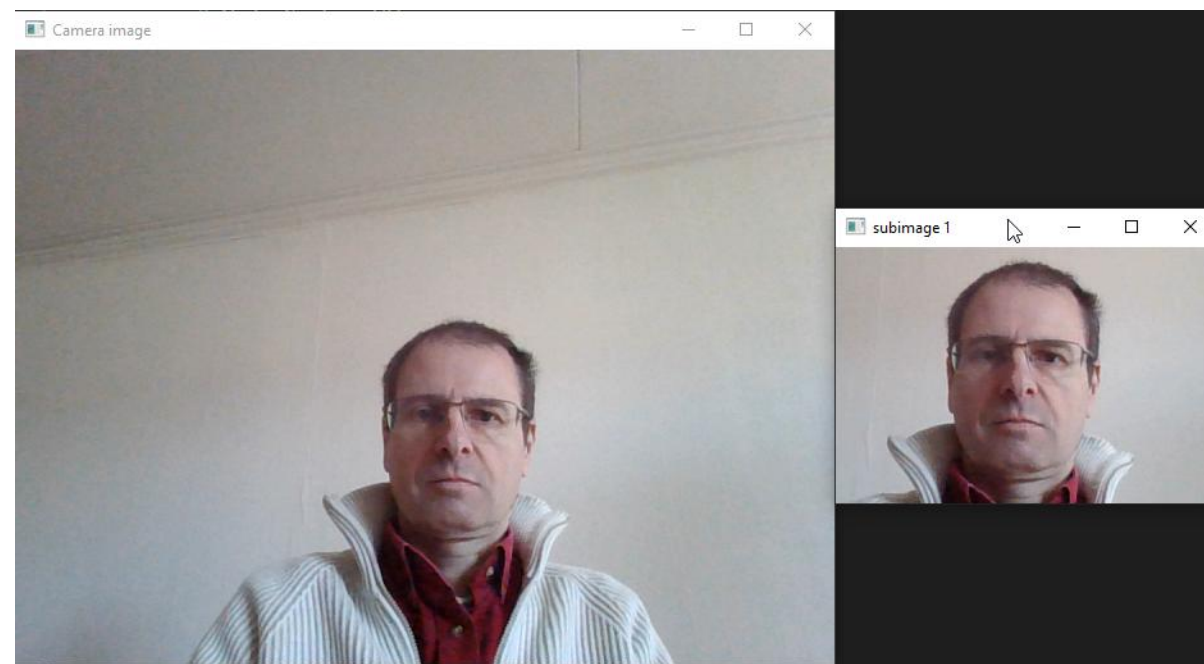
7) REGION OF INTEREST

```
improc-0070.py > ...
1  # we import all libraries used in our program
2  import cv2
3  import numpy
4  # Initialize Camera
5  cap = cv2.VideoCapture(0)
6  ret, frame = cap.read()
7  print("Shape: ", frame.shape)
8  nb = 0
9  # infinite loop
10 while True:
11     ret, frame = cap.read()
12     if ret:
13         frame = cv2.flip(frame, flipCode = 1)
14         frame1 = frame[0:frame.shape[0]//2, 0:frame.shape[1]//2]
15         frame2 = frame[frame.shape[0]//2:frame.shape[0], 0:frame.shape[1]//2]
16         frame3 = frame[0:frame.shape[0]//2, frame.shape[1]//2:frame.shape[1]]
17         frame4 = frame[frame.shape[0]//2:frame.shape[0], frame.shape[1]//2:frame.shape[1]]
18         if nb==0:
19             print("shapes 1,2,3,4: ", frame1.shape, frame2.shape, frame3.shape, frame4.shape)
20             nb += 1
21         frame1 = cv2.Canny(frame1, 100, 200)
22         frame2 = ((frame2>127)*255).astype(numpy.uint8)
23         frame3 = cv2.cvtColor(frame3, cv2.COLOR_BGR2GRAY)
24         frame4 = 255 - frame4
25         # we display it
26         cv2.imshow('Camera image', frame)
27         cv2.imshow('subimage 1', frame1)
28         cv2.imshow('subimage 2', frame2)
29         cv2.imshow('subimage 3', frame3)
30         cv2.imshow('subimage 4', frame4)
31         # we wait for a key pressed
32         k = cv2.waitKey(10)
33         # if it is 'q', we quit
34         if k == ord('q'):
35             break
36     # there is no image
37 else:
38     print('ERROR ---> Camera not found!')
39
```



7) REGION OF INTEREST

```
improc-0071.py > ...
1  # we import all libraries used in our program
2  import cv2
3  import numpy
4  # Initialize Camera
5  cap = cv2.VideoCapture(0)
6  ret, frame = cap.read()
7  # infinite loop
8  while True:
9      ret, frame = cap.read()
10     if ret:
11         frame = cv2.flip(frame, flipCode = 1)
12         frame1 = frame[200:400, 200:500]
13         # we display it
14         cv2.imshow('Camera image', frame)
15         cv2.imshow('subimage 1', frame1)
16         # we wait for a key pressed
17         k = cv2.waitKey(10)
18         # if it is 'q', we quit
19         if k == ord('q'):
20             break
21         # there is no image
22     else:
23         print('ERROR ---> Camera not found!')
24
25 # we close all windows
26 cv2.destroyAllWindows()
27
```



8) EXTRACT INFORMATION

- You can extract statistics from images to get information about the whole image and/or regions, ...
- Among all possible tools, the histogram of the image gives information about the distribution of colors in the image

8) EXTRACT INFORMATION

```

improc-0080.py > ...
1  # we import all libraries used in our program
2  import cv2
3  import numpy
4  # Initialize Camera
5  cap = cv2.VideoCapture(0)
6  ret, frame = cap.read()
7  # infinite loop
8  while True:
9      ret, frame = cap.read()
10     if ret:
11         frame = cv2.flip(frame, flipCode = 1)
12         mean = numpy.mean(frame)
13         min = numpy.min(frame)
14         max = numpy.max(frame)
15         stddev = numpy.std(frame)
16         print('Statistics: min, max, mean, stddev ---> ', min, max, mean, stddev)
17         # we display it
18         cv2.imshow('Camera image', frame)
19         # we wait for a key pressed
20         k = cv2.waitKey(10)
21         # if it is 'q', we quit
22         if k == ord('q'):
23             break
24     # there is no image
25     else:
26         print('ERROR ---> Camera not found!')
27
28 # we close all windows
29 cv2.destroyAllWindows()
30

```

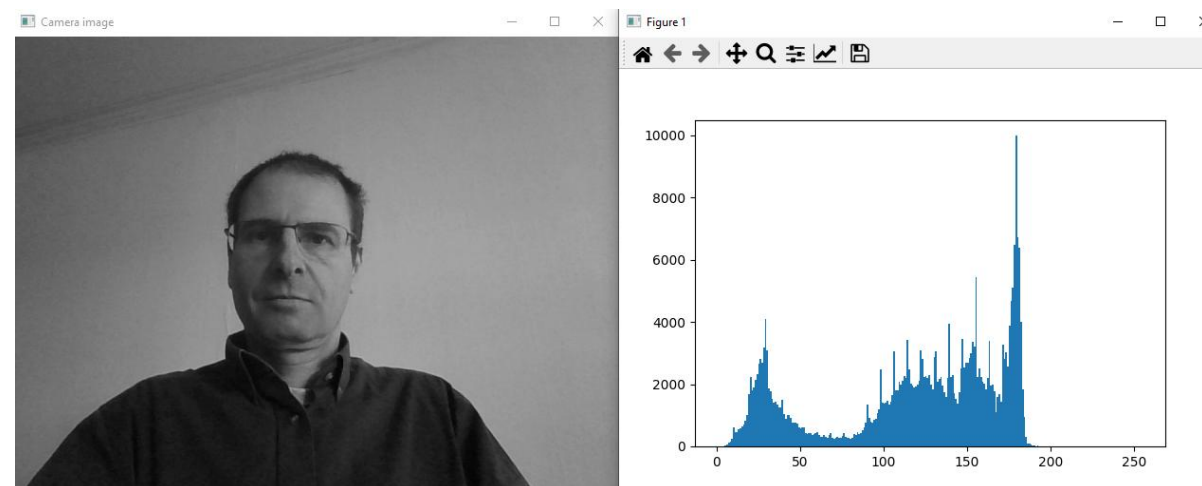
```

Statistics: min, max, mean, stddev ---> 0 221 121.81957139756945 33.391712692708815
Statistics: min, max, mean, stddev ---> 0 217 121.92159505208333 33.3514742402933
Statistics: min, max, mean, stddev ---> 0 221 121.96024848090278 33.34492241090537
Statistics: min, max, mean, stddev ---> 0 217 121.95812825520834 33.36773861152294
Statistics: min, max, mean, stddev ---> 0 223 121.9390603298611 33.327133106727736

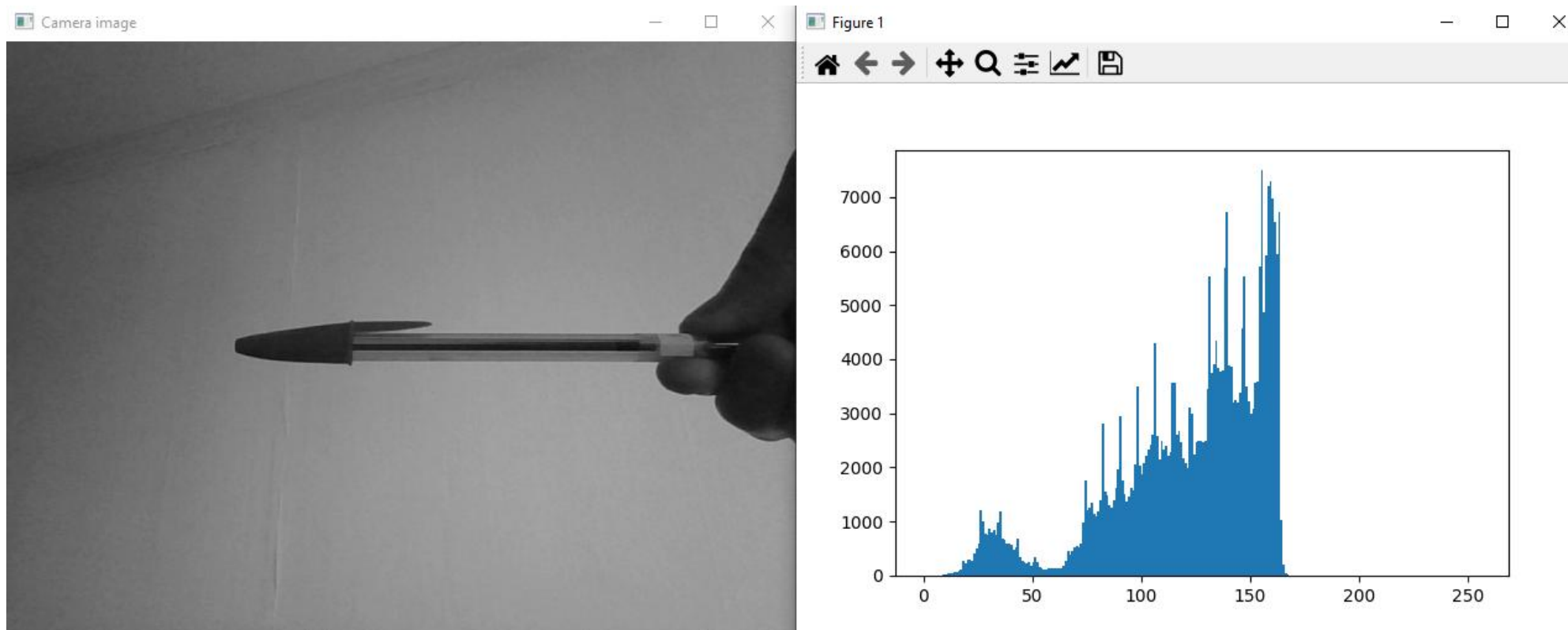
```

8) EXTRACT INFORMATION

```
improc-0081.py > ...
1  # we import all libraries used in our program
2  import cv2
3  import numpy
4  from matplotlib import pyplot as plt
5  # Initialize Camera
6  cap = cv2.VideoCapture(0)
7  ret, frame = cap.read()
8  # infinite loop
9  while True:
10     ret, frame = cap.read()
11     if ret:
12         frame = cv2.flip(frame, flipCode = 1)
13         frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
14         plt.hist(frame.ravel(), 256, [0, 256])
15         plt.show()
16         # we display it
17         cv2.imshow('Camera image', frame)
18         # we wait for a key pressed
19         k = cv2.waitKey(10)
20         # if it is 'q', we quit
21         if k == ord('q'):
22             break
23         # there is no image
24     else:
25         print('ERROR ---> Camera not found!')
26
27 # we close all windows
28 cv2.destroyAllWindows()
29
```

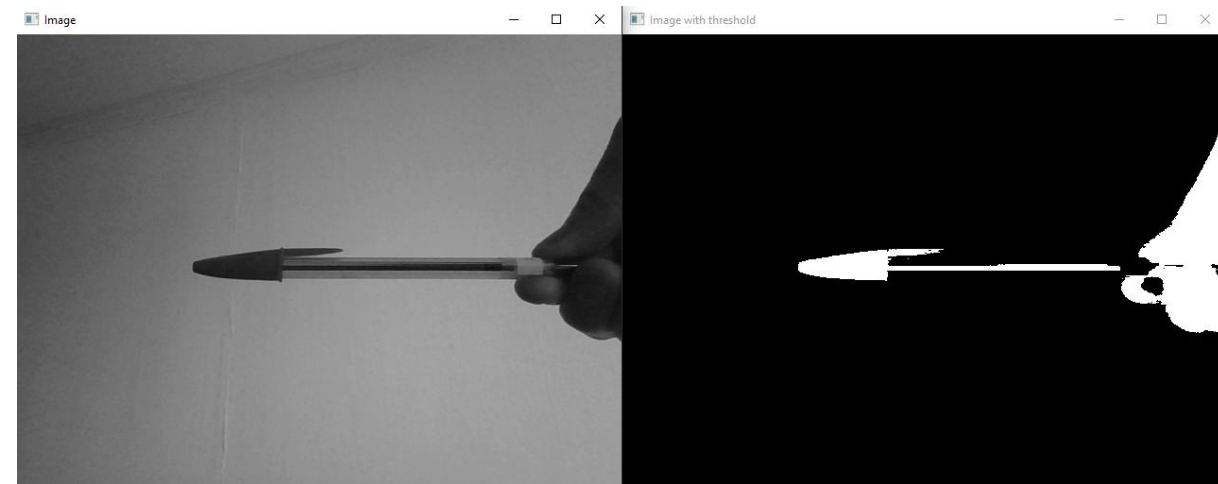


8) EXTRACT INFORMATION



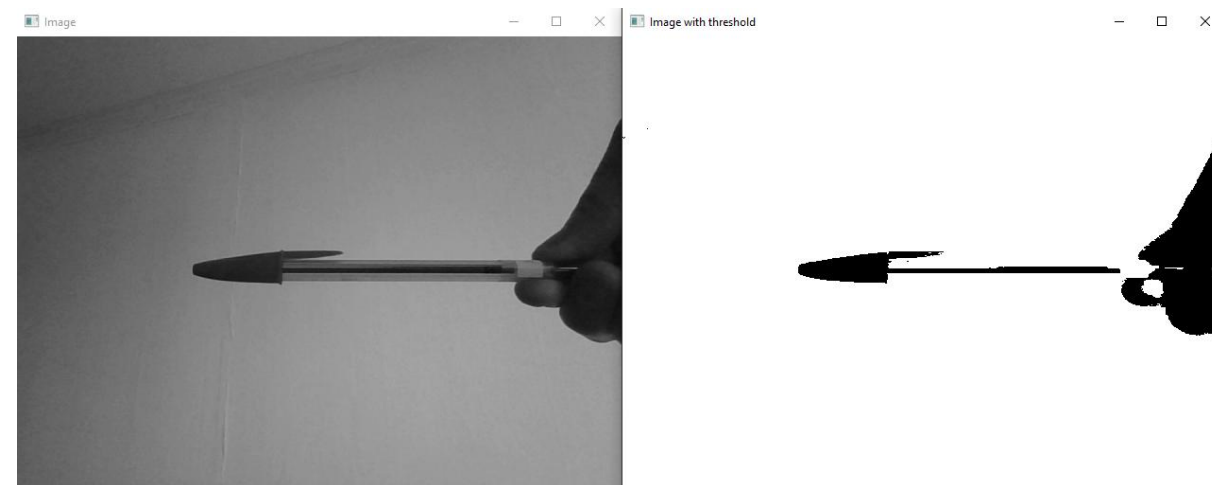
8) EXTRACT INFORMATION

```
improc-0082.py > ...
5  # we load the image
6  img = cv2.imread('penna.bmp')
7  print('Shape: ', img.shape)
8  # infinite loop
9  frame = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
10 plt.hist(frame.ravel(), 256, [0, 256])
11 plt.show()
12
13 frameThreshold = ((frame < 56).astype(numpy.uint8)) * 255
14 print(frameThreshold)
15 # we display it
16 cv2.imshow('Image', frame)
17 cv2.imshow('Image with threshold', frameThreshold)
18
19 # we wait for a key pressed
20 k = cv2.waitKey(0)
21
22 # we close all windows
23 cv2.destroyAllWindows()
24
```



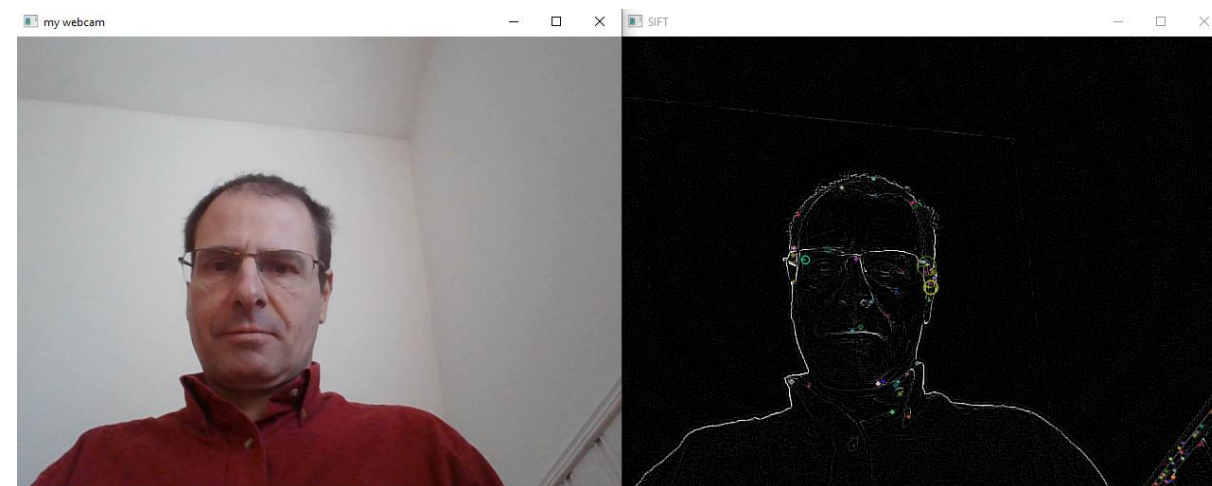
8) EXTRACT INFORMATION

```
improc-0083.py > ...
1  # we import all libraries used in our program
2  import cv2
3  import numpy
4  from matplotlib import pyplot as plt
5  # we load the image
6  img = cv2.imread('penna.bmp')
7  print('Shape: ', img.shape)
8  # infinite loop
9  frame = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
10 plt.hist(frame.ravel(), 256, [0, 256])
11 plt.show()
12
13 frameThreshold = ((frame > 56).astype(numpy.uint8)) * 255
14 print(frameThreshold)
15 # we display it
16 cv2.imshow('Image', frame)
17 cv2.imshow('Image with threshold', frameThreshold)
18
19 # we wait for a key pressed
20 k = cv2.waitKey(0)
21
22 # we close all windows
23 cv2.destroyAllWindows()
24
```



8) EXTRACT INFORMATION

```
improc-0110.py > show_webcam
1  import cv2
2  import numpy
3  def show_webcam(mirror=False):
4      ddepth = cv2.CV_8U
5      kernel_size = 3
6      cam = cv2.VideoCapture(0)
7      while True:
8          ret_val, img = cam.read()
9          imgSift = img.copy()
10         gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
11         grayLaplace = cv2.Laplacian(gray, ddepth, ksize=kernel_size)
12
13         # Applying SIFT detector
14         sift = cv2.SIFT_create()
15         kp, descriptor = sift.detectAndCompute(grayLaplace, None)
16
17         # Marking the keypoint on the image using circles
18         imgSift = cv2.drawKeypoints(grayLaplace,
19                                     kp,
20                                     imgSift,
21                                     flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
22
23         if mirror:
24             img = cv2.flip(img, 1)
25             imgSift = cv2.flip(imgSift, 1)
26         cv2.imshow('my webcam', img)
27         cv2.imshow('SIFT', imgSift)
28         if cv2.waitKey(1) == 27:
29             break # esc to quit
30     cv2.destroyAllWindows()
31
32 def main():
33     show_webcam(mirror=True)
34
35 if __name__ == '__main__':
36     main()
```



8) EXTRACT INFORMATION

```
improc-0111.py > ...
1  import cv2
2  def show_webcam(mirror=False):
3      cam = cv2.VideoCapture(0)
4      while True:
5          ret_val, img = cam.read()
6          imgSift = img.copy()
7          gray= cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
8          # Applying SIFT detector
9          sift = cv2.SIFT_create()
10         kp = sift.detect(gray, None)
11         # Marking the keypoint on the image using circles
12         imgSift=cv2.drawKeypoints(gray,
13                                 kp,
14                                 imgSift,
15                                 flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
16         if mirror:
17             img = cv2.flip(img, 1)
18             imgSift = cv2.flip(imgSift, 1)
19         cv2.imshow('my webcam', img)
20         cv2.imshow('SIFT', imgSift)
21         if cv2.waitKey(1) == 27:
22             break # esc to quit
23     cv2.destroyAllWindows()
24 def main():
25     show_webcam(mirror=True)
26 if __name__ == '__main__':
27     main()
28
```

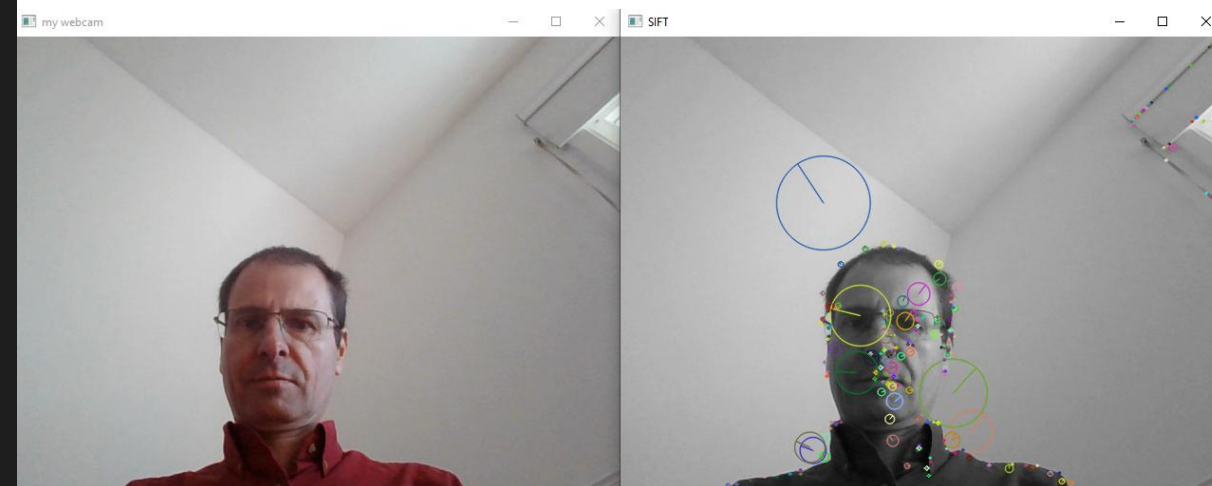


Image registration:

<https://learnopencv.com/image-alignment-feature-based-using-opencv-c-python/>

9) MACHINE LEARNING

- In order to find objects in images, you can use pre-trained machine learning algorithms (or train them yourself)
- To use these tools, you must have a powerful computer (GPU processing)
- In order to boost performances, you must program in C++ instead of Python

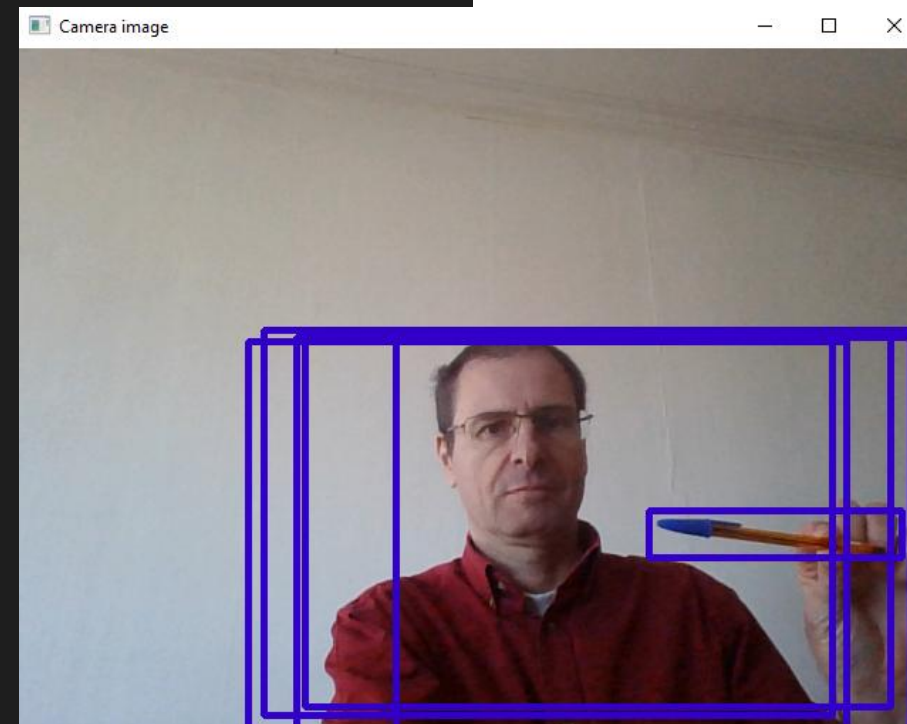
9) MACHINE LEARNING (YOLO V3)

improc-0099-model.py > ...

```

1  import cv2
2  # OpenCV DNN
3  #source: https://github.com/patrick013/Object-Detection---Yolov3/blob/master/model/yolov3.weights
4  #another source: https://learnopencv.com/deep-learning-based-object-detection-using-yolov3-with-opencv-python-c/
5  net = cv2.dnn.readNet('model/yolov3.weights', 'model/yolov3.cfg')
6  net.setPreferableBackend(cv2.dnn.DNN_BACKEND_CUDA)
7  net.setPreferableTarget(cv2.dnn.DNN_TARGET_CUDA_FP16)
8  model = cv2.dnn_DetectionModel(net)
9  model.setInputParams(size=(320, 320), scale=1/255, swapRB=True)
10 # Initialize Camera
11 cap = cv2.VideoCapture(0)
12 while True:
13     ret, frame = cap.read()
14     if ret:
15         (class_ids, scores, bboxes) = model.detect(frame)
16         for class_id, score, bbox in zip(class_ids, scores, bboxes):
17             (x, y, w, h) = bbox
18             cv2.rectangle(frame, (x, y), (x+w, y+h), (200, 0, 50), 3)
19         cv2.imshow('Camera image', frame)
20         k = cv2.waitKey(100)
21         if k==ord('q'):
22             break
23     else:
24         print('ERROR ---> Camera not found!')
25
26 cv2.destroyAllWindows()

```

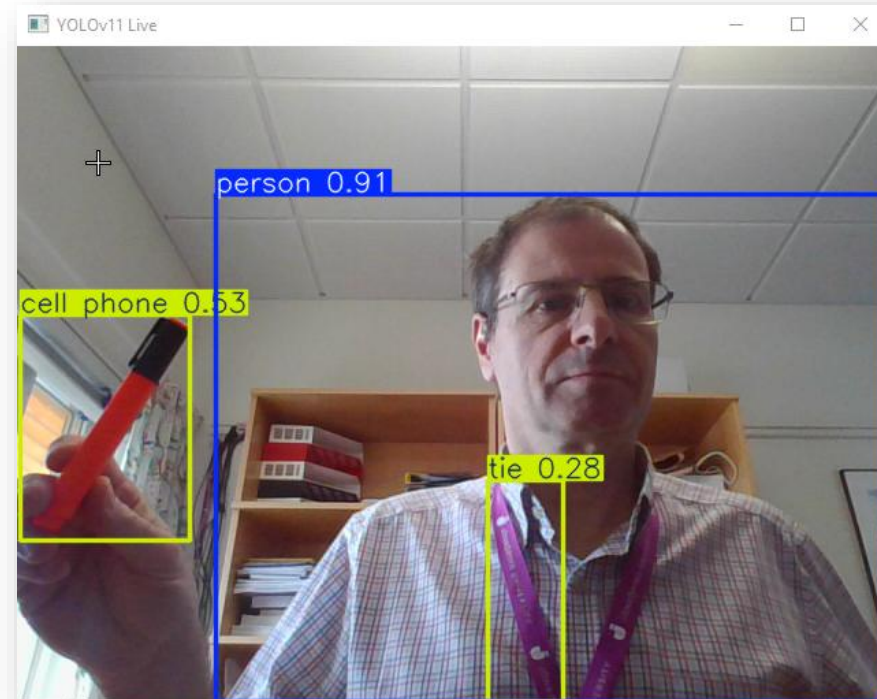


9) MACHINE LEARNING (YOLO V11)

```

1  import cv2
2  # Using ultralytics YOLOv11: https://github.com/ultralytics/ultralytics
3  # YOU MUST RUN: pip install ultralytics
4  from ultralytics import YOLO
5
6  # Load a pretrained YOLO11n model
7  model = YOLO("yolo11n.pt")
8
9  # Initialize Camera
10 cap = cv2.VideoCapture(0)
11 while cap.isOpened():
12     success, frame = cap.read()
13     if not success:
14         break
15     results = model(frame, imgsz=640) # detect objects
16     annotated_frame = results[0].plot() # plot the results
17     cv2.imshow("YOLOv11 Live", annotated_frame) # display the annotated image
18     if cv2.waitKey(1) & 0xFF == ord("q"):
19         break
20
21 cap.release() # release the camera
22 cv2.destroyAllWindows() # destroy all windows
23

```



```

0: 480x640 1 person, 1 toothbrush, 96.0ms
Speed: 1.8ms preprocess, 96.0ms inference, 1.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 1 toothbrush, 79.1ms
Speed: 1.7ms preprocess, 79.1ms inference, 1.2ms postprocess per image at shape (1, 3, 480, 640)

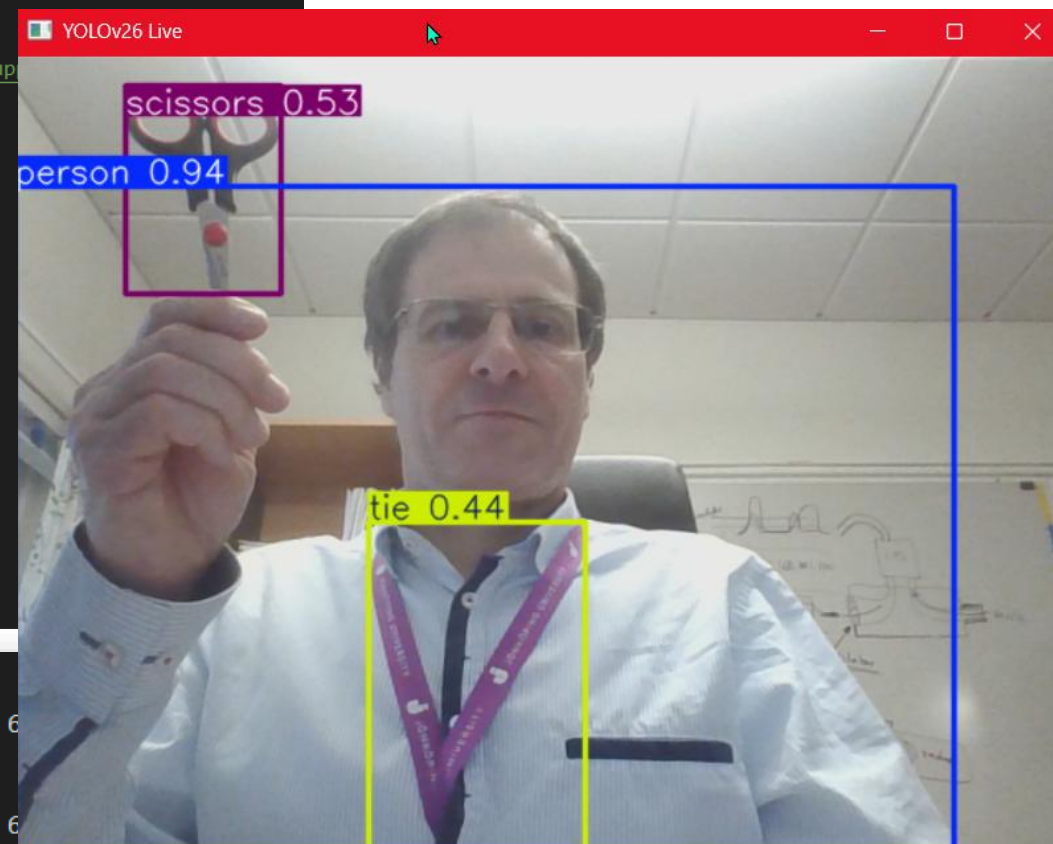
```

9) MACHINE LEARNING (YOLO V26)

```

1 import cv2
2 # Using ultralytics YOLOv26: https://docs.ultralytics.com/models/yolo26 and https://github.com/ultralytics/ultralytics
3 # YOU MUST RUN: pip install ultralytics
4 from ultralytics import YOLO
5
6 # Load a pretrained YOLO26 model according to your needs: https://docs.ultralytics.com/models/yolo26/#sup
7 model = YOLO("yolo26n.pt")
8 # model with segmentation: model = YOLO("yolo26-seg.pt")
9
10 # Initialize Camera
11 cap = cv2.VideoCapture(0)
12 while cap.isOpened():
13     success, frame = cap.read()
14     if not success:
15         break
16     results = model(frame, imgsz=640) # detect objects
17     annotated_frame = results[0].plot() # plot the results
18     cv2.imshow("YOLOv26 Live", annotated_frame) # display the annotated image
19     if cv2.waitKey(1) & 0xFF == ord("q"):
20         break
21
22 cap.release() # release the camera
23 cv2.destroyAllWindows() # destroy all windows
24

```



```

0: 480x640 1 person, 1 tie, 1 book, 165.1ms
Speed: 3.9ms preprocess, 165.1ms inference, 0.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 1 tie, 1 book, 151.0ms
Speed: 2.5ms preprocess, 151.0ms inference, 0.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 1 tie, 1 book, 199.0ms
Speed: 36.8ms preprocess, 199.0ms inference, 0.7ms postprocess per image at shape (1, 3, 480, 640)

```

10) TOOLS

- You can get information about your OpenCV version
- You can also use keys and mouse events onto your images

10) TOOLS

```

1  import cv2
2  import datetime
3  print("---> OpenCV version : {0}".format(cv2.__version__))
4  # Initialize Camera
5  cap = cv2.VideoCapture(0)
6  # infinite loop
7  while True:
8      # we read an image from the camera
9      ret, frame = cap.read()
10     # there is an image
11     if ret:
12         frame = cv2.flip(frame, flipCode = 1)
13         # we display it
14         cv2.imshow('Camera image', frame)
15         # we wait for a key pressed
16         k = cv2.waitKey(10)
17         # if it is 'q', we quit
18         if k == ord('q'):
19             break
20         if k == ord('s'):
21             the_date = datetime.datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
22             filenameRGB = 'outRGB---'+the_date+'.png'
23             cv2.imwrite(filenameRGB, frame)
24             print('Image ', filenameRGB, ' saved...')
25     # there is no image
26     else:
27         print('ERROR ---> Camera not found!')
28
29  cap.release() # release the camera
30  cv2.destroyAllWindows() # destroy all windows

```



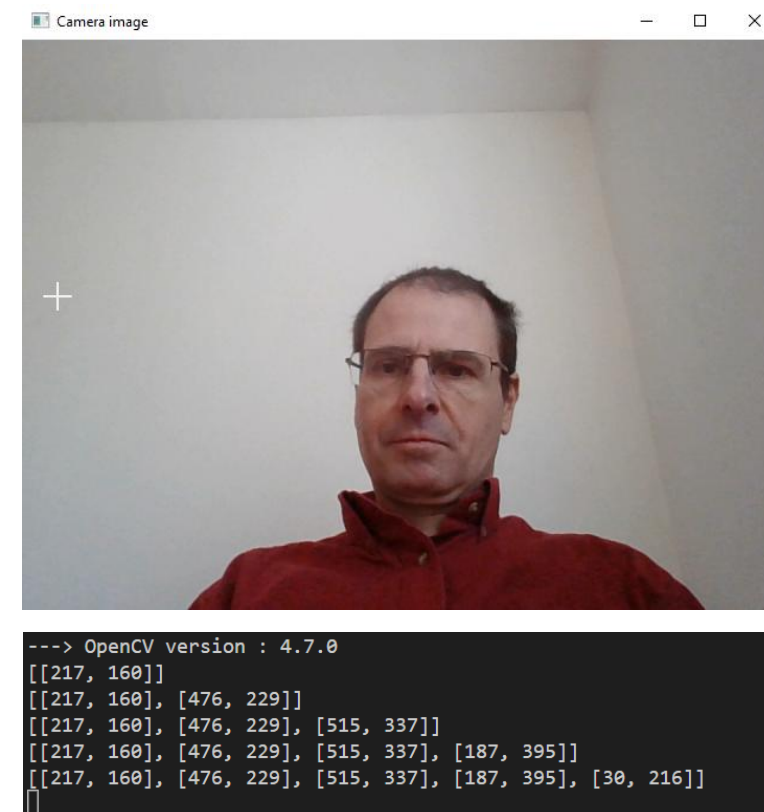
```

---> OpenCV version : 4.9.0
Image  outRGB---2025-04-11_09-33-30.png  saved...
Image  outRGB---2025-04-11_09-33-48.png  saved...

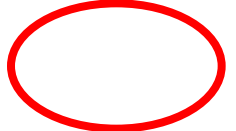
```

10) TOOLS

```
improc-0101.py > ...
2 import cv2
3 import datetime
4 print("---> OpenCV version : {0}".format(cv2.__version__))
5 #source: https://thinkinfi.com/detect-mouse-click-events-with-python-opencv/
6 listPoints = []
7 def mousePoints(event,x,y,flags,params):
8     global listPoints
9     # Left button mouse click event opencv
10    if event == cv2.EVENT_LBUTTONDOWN:
11        listPoints.append([x, y])
12        print(listPoints)
13    # Initialize Camera
14    cap = cv2.VideoCapture(0)
15    # infinite loop
16    while True:
17        ret, frame = cap.read()
18        if ret:
19            frame = cv2.flip(frame, flipCode = 1)
20            cv2.imshow('Camera image', frame)
21            cv2.setMouseCallback("Camera image", mousePoints)
22            k = cv2.waitKey(10)
23            if k == ord('q'):
24                break
25            if k == ord('s'):
26                x = datetime.datetime.now()
27                filenameRGB = 'outRGB'+str(x.year)+str(x.month)+str(x.day)+str(x.hour)+str(x.minute)+str(x.second)+'.png'
28                cv2.imwrite(filenameRGB, frame)
29                print('Image ', filenameRGB, ' saved...')
30            # there is no image
31        else:
32            print('ERROR ---> Camera not found!')
33    # we close all windows
34    cv2.destroyAllWindows()
```



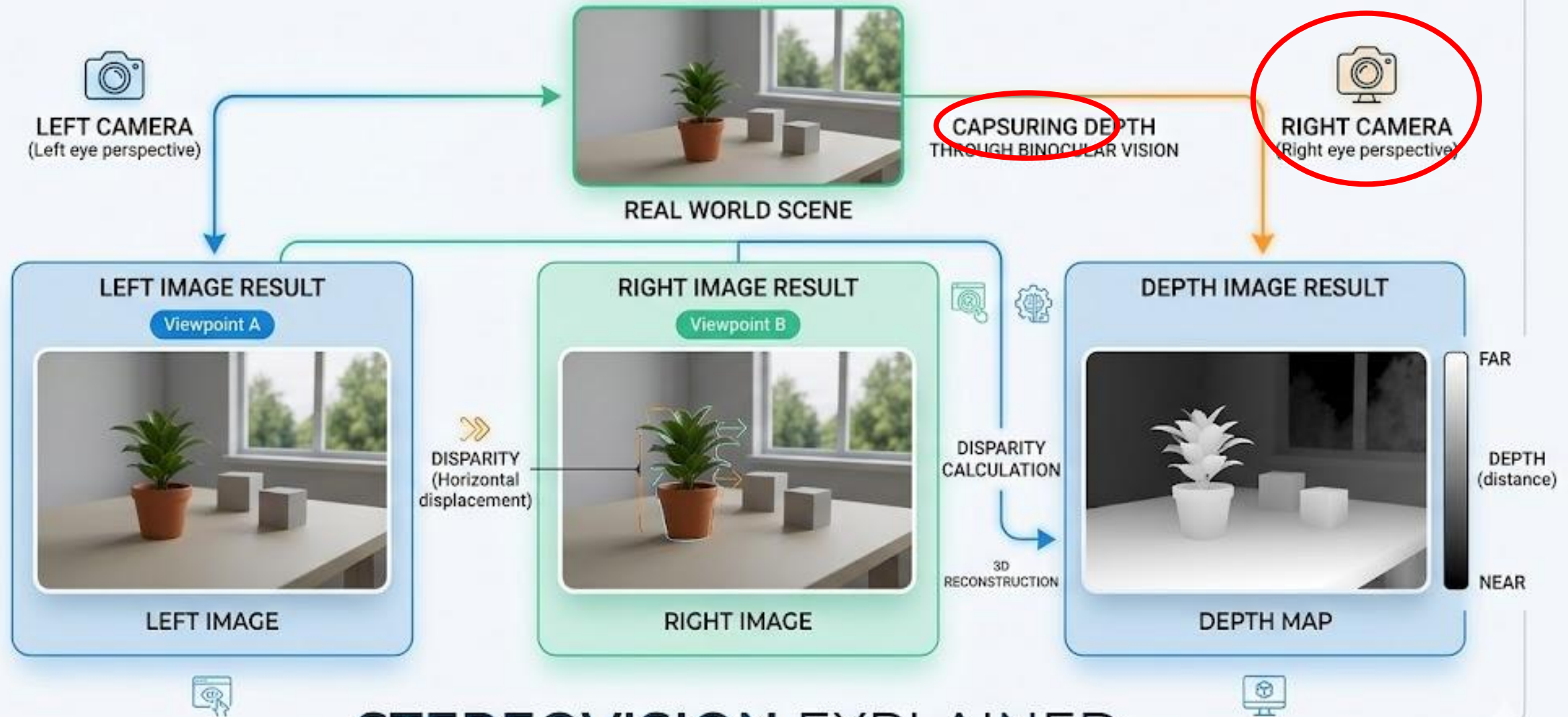
11) OTHER (IMAGE) SENSORS

- All the images in this section were generated using Google Gemini, <https://gemini.google.com/>
- The result is very good, but I found several hallucinations: 

⇒ **DO NOT TRUST AI 100%**

⇒ **Always check the results!**

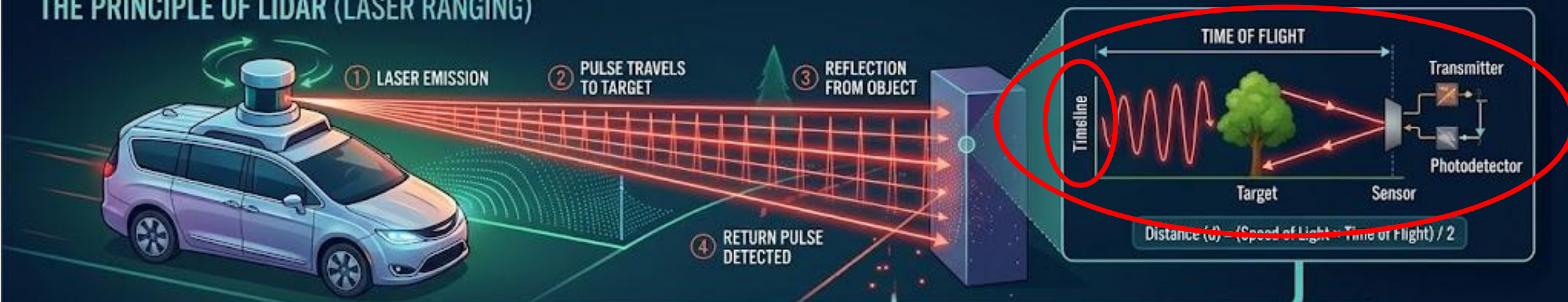
STEREOVISION EXPLAINED



STEREOVISION EXPLAINED

UNDERSTANDING LIDAR: HOW LIGHT DETECTION AND RANGING WORKS AND ITS APPLICATIONS

THE PRINCIPLE OF LIDAR (LASER RANGING)



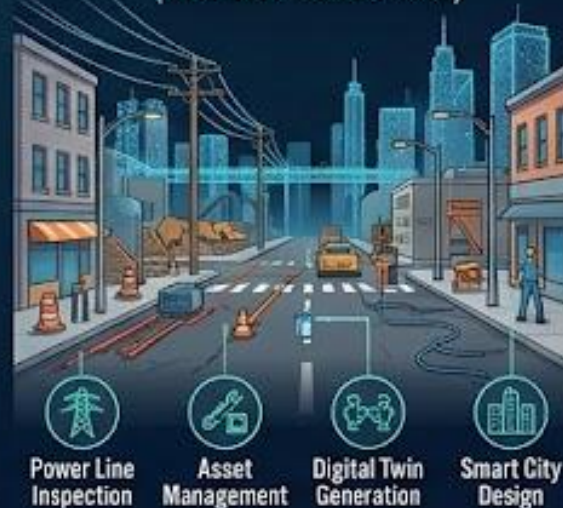
AUTONOMOUS VEHICLES & ROBOTICS



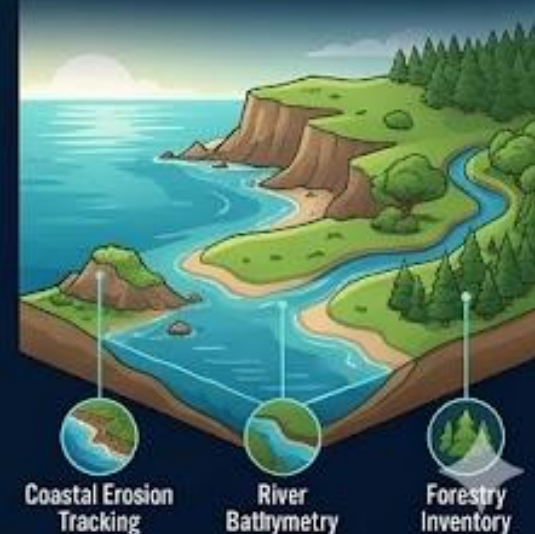
TERRAIN MAPPING & SURVEYING (AERIAL LIDAR)



INFRASTRUCTURE & URBAN PLANNING (MOBILE & TERRESTRIAL)

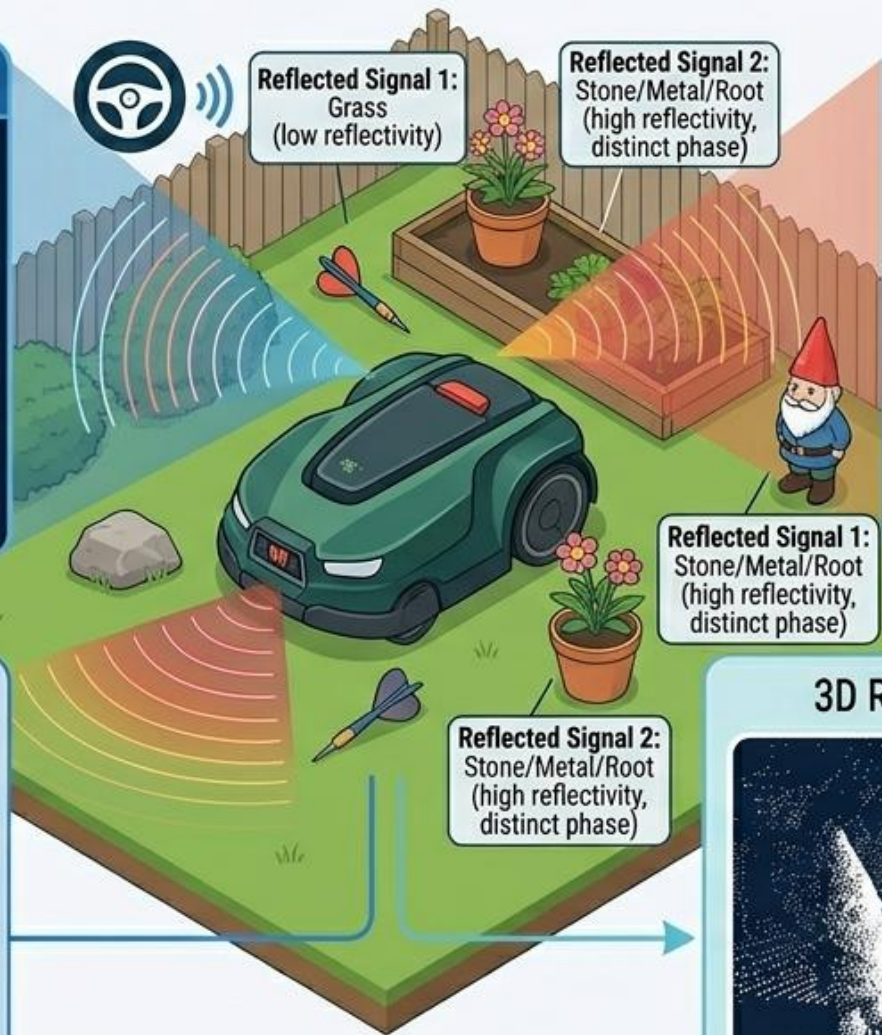
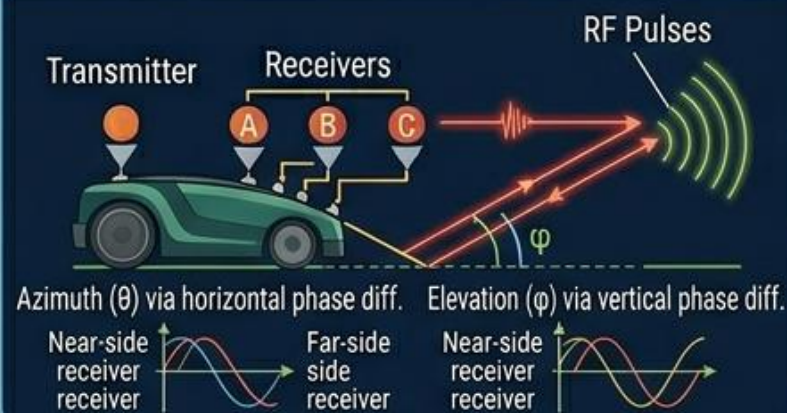


ENVIRONMENTAL MONITORING

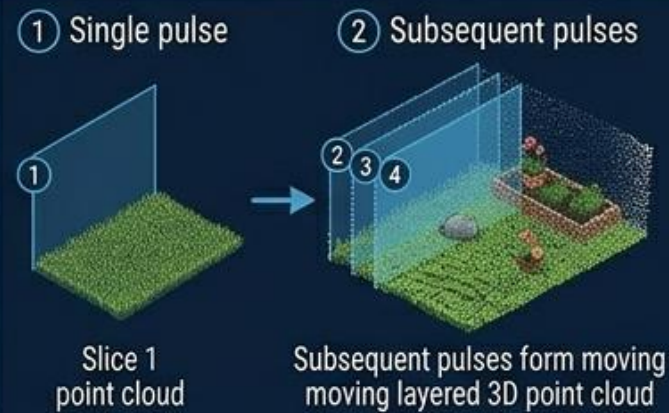


AUTONOMOUS LAWN MOWER 3D RADAR MAPPING

RF PULSE & PHASE MEASUREMENT (3D RANGING)



3D POINT CLOUD ACCUMULATION

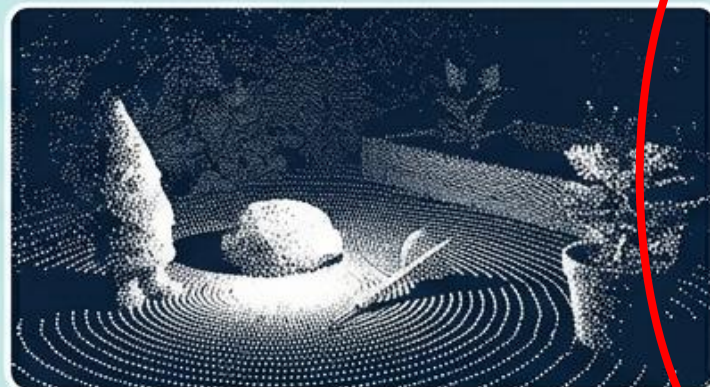


ACTUAL GARDEN VIEW (REFERENCE)



LEFT IMAGE (REFERENCE)

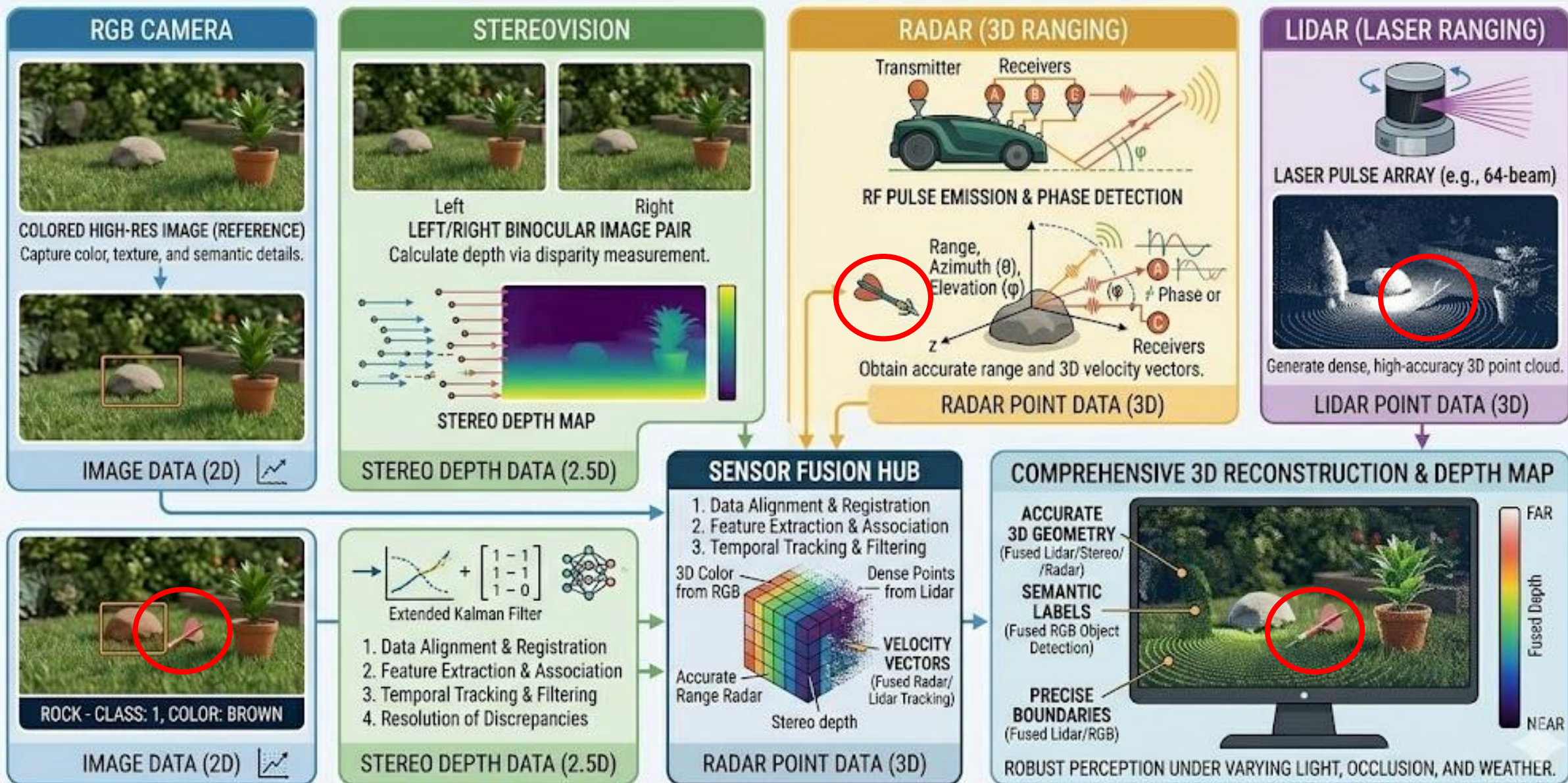
3D RECONSTRUCTED POINT CLOUD MAP



RECONSTRUCTED ENVIRONMENT (Viewpoint A)

ROBOTIC LAWN MOWER
MAPPING SYSTEMS

MULTI-SENSOR INFORMATION FUSION FOR ROBUST 3D PERCEPTION



MORE INFORMATION

- Official OpenCV documentation: <https://docs.opencv.org/>
- GeeksforGeeks OpenCV Python tutorial: <https://www.geeksforgeeks.org/opencv-python-tutorial/>
- Learn OpenCV: <https://learnopencv.com>
- PyImageSearch: <https://pyimagesearch.com/>



JÖNKÖPING UNIVERSITY

School of Engineering